CANE — Context-Aware Notification Engine

LinUCB (contextual-bandit baseline)

This notebook implements the **LinUCB** arm of the CANE comparison.

LinUCB is the deliberately *stateless* baseline in the algorithm progression (bandit -> DQN -> Double DQN -> PPO). Its role is to answer **RQ2**: does a contextual bandit, which optimises only the immediate expected reward, suffice for fatigue-aware notification pacing — or is a full MDP method genuinely required?

Build order

Section	Contents
1	Setup — imports, action constants
2	Agent interface (shared contract for all four algorithms)
3	Feature encoders (Raw / Harmonic / One-hot)
4	LinUCB agent
5	Mock environment (placeholder until the real CANE env lands)
6	Synthetic correctness test
7	Experiments — alpha sweep, feature ablation, per-archetype results

---

1. Setup

LinUCB needs only `numpy` — it is closed-form linear algebra, with no neural network and no autograd. `pandas` and `matplotlib` are for the analysis sections.

State contract

Two decisions here shape everything downstream, so they are recorded explicitly:

1. The observation is a flat `float32` array, not a dict. This keeps it a standard `gymnasium` `Box` space, which stable-baselines3 consumes directly — relevant because the DQN and PPO arms of the comparison rely on it. Index constants restore readability: `state[IDX_FATIGUE]` rather than `state[3]`.

2. Hour is emitted raw (0–23), *not* pre-encoded as sin/cos. Encoding is the agent's job. This is a hard requirement for the feature ablation in Section 3: once the raw hour has been collapsed into sin/cos, the Harmonic and One-hot encodings can no longer be reconstructed from it.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from abc import ABC, abstractmethod


# --- Action space ------------------------------------------------------------
# Defined once and imported everywhere so the agents, the environment and the
# plotting code can never disagree about what "1" means.
HOLD = 0        # stay silent -> fatigue decays
ENGAGE = 1      # engagement nudge (social / community update)
INCENTIVE = 2   # incentive nudge (high-value promo reward)

N_ACTIONS = 3
ACTION_NAMES = {HOLD: "Hold", ENGAGE: "Engage", INCENTIVE: "Incentive"}


# --- Observation layout ------------------------------------------------------
# The env returns s = [hour, day, active, fatigue, recency] as a flat float32
# array. These constants are the single source of truth for the column order;
# if the env owner reorders the vector, only this block changes.
IDX_HOUR = 0        # 0-23, RAW (not sin/cos) - see the note above
IDX_DAY = 1         # 0-6, Monday = 0
IDX_ACTIVE = 2      # 1 if the user is currently in-app
IDX_FATIGUE = 3     # [0, 1] accumulated notification fatigue
IDX_RECENCY = 4     # [0, 1] normalised time since the last send

N_BLOCKS = 8                # 3-hour blocks; separates the 19:00 and 23:00 peaks
BLOCK_HOURS = 24 // N_BLOCKS

IDX_ACT_RATE = 5            # 5..12  smoothed in-app activity rate per block
IDX_CLICK_RATE = 13         # 13..20 smoothed click rate per block
IDX_TYPE_CR = 21            # 21,22  click rate for Engage / Incentive sends
IDX_SINCE_CLICK = 23        # normalised hours since the last click

STATE_DIM_BASE = 5
STATE_DIM = 24

BELIEF_PRIOR = (1.0, 4.0)   # Beta(a, b); prior mean 0.2 before any evidence
USE_BELIEF = True           # False reproduces the memoryless 5-feature ablation


def agent_state_dim():
    return STATE_DIM if USE_BELIEF else STATE_DIM_BASE

print("Actions:", ACTION_NAMES)
print("State dim:", STATE_DIM, "| agent sees:", agent_state_dim())
```

```
Actions: {0: 'Hold', 1: 'Engage', 2: 'Incentive'}
State dim: 24 | agent sees: 24
```

---

2. Agent interface

All four algorithms implement this one contract, so a single evaluation harness can run any of them without special-casing.

Why it is shaped around PPO rather than LinUCB. LinUCB has the narrowest requirements of the four — it only ever needs `(state, action, reward)`. An interface designed around it would break the moment the other three were added, so it is designed around the most demanding consumer instead:

Element	Who needs it
`aux` returned by `act()`	PPO — the `log_prob` must be captured *when the action is sampled*; recomputing it later gives the wrong value because the policy has since moved. LinUCB returns `{}`.
`next_state`, `done` in `update()`	DQN / Double DQN — bootstrapping `r + γ·max Q(s')` needs both. LinUCB ignores them.
`greedy` flag	Everyone. All reported results use greedy evaluation: LinUCB drops its UCB bonus, DQN sets ε = 0, PPO takes the mode instead of sampling.
dict returned by `update()`	The logger, so training diagnostics are recorded uniformly across algorithms.

Accepting arguments it does not use costs LinUCB nothing, and it means the same harness, logger and plots serve the whole team.

In [2]:
```python
# --- The agent contract -----------------------------------------------------
# Every policy in this notebook -- the two non-learning baselines and the
# learner alike -- implements this interface, which is what lets the evaluation
# harness below treat them all identically. The comparison is only like-for-like
# because no agent gets a code path of its own.
#
# `act` returns (action, aux). The aux dict is how an agent hands the harness
# whatever it needs to record: the deep agents put their Q-values in it.
# `update` is a no-op for the baselines; `reset` is called at every episode
# boundary, which matters for any agent holding per-episode state.
class Agent(ABC):
    """Shared contract implemented by every CANE agent.

    `act` and `update` are abstract: a subclass that omits either one cannot be
    instantiated, so an incomplete agent fails loudly at construction rather
    than silently producing a policy that never learns.
    """

    @property
    def name(self):
```

```
        """Label used in results tables and plot legends.

        Defaults to the class name. LinUCB overrides this, because the feature
        variants (Raw / Harmonic / One-hot) are the same class and would
        otherwise collapse into a single indistinguishable row.
        """
        return self.__class__.__name__

    @abstractmethod
    def act(self, state, greedy=False):
        """Choose an action for the given state.

        Args:
            state: observation array from the env (see the index constants).
            greedy: act without exploration. Used for all reported evaluation
                runs, so the numbers reflect the learned policy rather than
                exploration noise.

        Returns:
            (action, aux). `action` is an int in {HOLD, ENGAGE, INCENTIVE}.
            `aux` carries any per-step quantity the agent will need back at
            update time, and is empty for agents that need nothing.
        """
        raise NotImplementedError

    @abstractmethod
    def update(self, state, action, reward, next_state, done, aux):
        """Learn from one transition.

        The full transition is always passed and each agent takes what it
        needs; LinUCB uses only `state`, `action` and `reward`.

        Returns:
            dict of diagnostics for the logger (may be empty).
        """
        raise NotImplementedError

    def reset(self):
        """Called at the start of each episode. No-op by default.

        LinUCB must NOT reset here: its A and b matrices accumulate across
        every episode, and that accumulation is the whole of its learning.
        Clearing them per episode would silently reduce it to a random policy.
        On-policy agents such as PPO do override this, to clear their buffer.
        """
        pass


print("Agent interface defined.")
```

```
Agent interface defined.
```

---

3. Feature encoders

LinUCB is a **linear** model, so what it is capable of representing is decided here, not in the learning rule. This section is therefore not boilerplate — it is the core of the RQ2 argument.

Why three variants

Consider the encoding the proposal specifies, `(sin h, cos h)`. A linear function of those two terms is `a·sin h + b·cos h`, which is a single sinusoid: **exactly one peak and one trough per day**. Now compare that against the archetypes the environment defines — the Office Worker is receptive at the morning commute, at lunch, *and* in the evening (three peaks); the Night-Shift Worker is bimodal.

A model that can only draw one hump per day cannot fit any of them. Reported on its own, a poor LinUCB score would therefore be uninterpretable: is the bandit formulation inadequate, or was the *encoding* simply too weak to express the answer? The three variants separate those two explanations.

Variant	Time features	d	What it tests
`raw`	`sin h, cos h`	7	the proposal exactly as written — one daily peak
`harmonic`	`sin/cos` of `h, 2h, 3h`	11	can multi-peak timing be recovered with a richer basis?
`onehot`	24 hour indicators	29	no functional-form constraint at all — any hourly pattern is representable

The `onehot` variant matters most. It gives LinUCB *complete* freedom over time-of-day, so if it still trails the deep methods, the shortfall cannot be blamed on representation. That is the strong form of the RQ2 claim, and it also protects the comparison from the obvious criticism that the baseline was handicapped.

Design choices

Day of week becomes a weekday/weekend binary. Feeding the integer 0–6 into a linear model asserts that Sunday is "six times" Monday, which is meaningless. One-hot over seven days would spend six parameters to capture what is, in these archetypes, a single distinction: whether the routine shifts at the weekend.

A constant `1.0` (intercept) is included. This lets each arm learn its own baseline value — necessary here, because `HOLD` has a genuinely different average reward from the two send actions. The features are deliberately *not* also mean-centred, which would encode the same information twice and make the design matrix ill-conditioned.

No interaction terms yet. The ablation isolates one thing: time-representation capacity. Adding `F × hour` at the same time would confound which change caused any improvement. If `onehot` still underperforms, interactions become the natural follow-up experiment.

All features sit on a comparable scale — `sin/cos` and the indicators in [-1, 1], `fatigue` and `recency` already normalised to [0, 1]. This matters because the ridge term penalises every coefficient equally; wildly different feature scales would make that penalty arbitrary.

In [3]:

```
def _harmonics(hour, n_harmonics):
    """sin/cos pairs for the first `n_harmonics` harmonics of the 24h cycle.

    The k-th harmonic completes k full cycles per day, so including k = 1..3
    lets a linear model express up to three peaks per day instead of one.
    """
    out = []
    for k in range(1, n_harmonics + 1):
        theta = 2.0 * np.pi * k * hour / 24.0
        out.extend([np.sin(theta), np.cos(theta)])
    return out


def _shared_tail(state):
    """The non-temporal features, identical across all three variants.

    Keeping these in one place is what makes the ablation clean: the ONLY
    difference between the variants is how time-of-day is encoded.
    """
    feats = [
        1.0 if state[IDX_DAY] >= 5 else 0.0,   # weekend (Sat = 5, Sun = 6)
        float(state[IDX_ACTIVE]),
        float(state[IDX_FATIGUE]),
        float(state[IDX_RECENCY]),
    ]
    if USE_BELIEF:
        feats += list(np.asarray(state[IDX_ACT_RATE:IDX_ACT_RATE + N_BLOCKS], dtype
        feats += list(np.asarray(state[IDX_CLICK_RATE:IDX_CLICK_RATE + N_BLOCKS], d
        feats += [float(state[IDX_TYPE_CR]), float(state[IDX_TYPE_CR + 1])]
        feats.append(float(state[IDX_SINCE_CLICK]))
    feats.append(1.0)                           # intercept
    return feats


def encode_raw(state):
    """Variant 1 - the proposal as written. d = 7.

    A linear model over (sin h, cos h) is a single sinusoid, so this can
    represent exactly one receptive period per day.
    """
    feats = _harmonics(state[IDX_HOUR], 1) + _shared_tail(state)
    return np.asarray(feats, dtype=np.float64)


def encode_harmonic(state):
    """Variant 2 - richer Fourier basis. d = 11.

    Harmonics up to k = 3 can express up to three peaks per day, which is what
    the Office Worker archetype (commute / lunch / evening) actually requires.
    """
```

```
    feats = _harmonics(state[IDX_HOUR], 3) + _shared_tail(state)
    return np.asarray(feats, dtype=np.float64)


def encode_onehot(state):
    """Variant 3 - unconstrained hourly indicators. d = 29.

    Each hour gets its own free parameter, so ANY hourly pattern is
    representable. This is the variant that decides whether LinUCB's shortfall
    is representational or structural.
    """
    hours = [0.0] * 24
    hours[int(state[IDX_HOUR]) % 24] = 1.0
    feats = hours + _shared_tail(state)
    return np.asarray(feats, dtype=np.float64)


def encode_interaction(state):
    """Variant 4 - hour indicators crossed with the belief block.

    A linear model over additive features cannot express "send at 23:00 IF this
    user's late-evening activity is high": that rule is a product of two
    features. Supplying the products explicitly removes the hypothesis class as
    a confound, so any remaining shortfall against the deep methods is
    attributable to the bandit formulation rather than to the encoding.
    """
    hours = np.zeros(24)
    hours[int(state[IDX_HOUR]) % 24] = 1.0
    feats = list(hours)
    if USE_BELIEF:
        act = np.asarray(state[IDX_ACT_RATE:IDX_ACT_RATE + N_BLOCKS], dtype=float)
        feats += list(np.outer(hours, act).ravel())
    feats += _shared_tail(state)
    return np.asarray(feats, dtype=np.float64)


# --- Registry ----------------------------------------------------------------
# Lets experiments select an encoding by name from config, so the ablation is a
# loop over keys rather than three near-duplicate code paths.
FEATURE_ENCODERS = {
    "raw": encode_raw,
    "harmonic": encode_harmonic,
    "onehot": encode_onehot,
    "interaction": encode_interaction,
}


def feature_dim(encoder):
    """Infer d by probing the encoder with a dummy state.

    Derived rather than hard-coded: if an encoder is later edited, d follows
    automatically instead of silently disagreeing with a stale constant.
    """
    probe = np.zeros(STATE_DIM, dtype=np.float32)
    return len(encoder(probe))
```

```
for _name, _enc in FEATURE_ENCODERS.items():
    print(f"{_name:>9}: d = {feature_dim(_enc)}")
```

```
      raw: d = 26
 harmonic: d = 30
   onehot: d = 48
interaction: d = 240
```

In [4]:

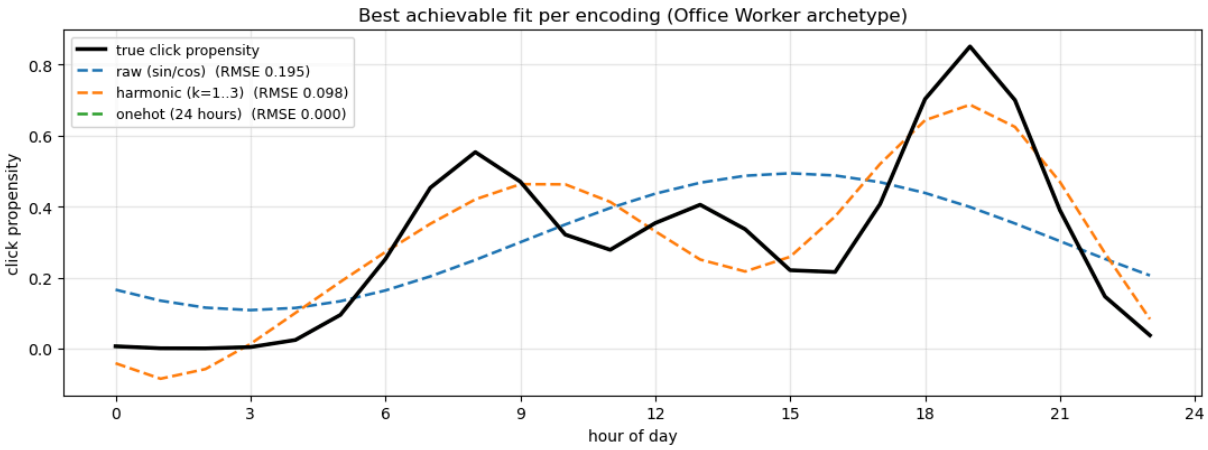
```
# --- Verification ------------------------------------------------------------
# Cheap invariants, checked now so that a malformed feature vector cannot be
# mistaken later for a failure of the bandit itself.

def _make_state(hour, day=0, active=0, fatigue=0.0, recency=0.0):
    """Build a synthetic observation for testing."""
    # The belief state. Note what is absent: the archetype, the true
    # receptiveness curve, and the click probability. The agent gets only
    # what an app could actually log about a user.
    s = np.zeros(STATE_DIM, dtype=np.float32)
    s[IDX_HOUR], s[IDX_DAY] = hour, day
    s[IDX_ACTIVE], s[IDX_FATIGUE], s[IDX_RECENCY] = active, fatigue, recency
    return s


# Derived from each encoder's documented structure rather than hard-coded, so
# the check still fails loudly if an encoder is wrong but survives a change to
# the state layout.
_tail = len(_shared_tail(_make_state(0)))
expected_dims = {
    "raw": 2 + _tail,
    "harmonic": 6 + _tail,
    "onehot": 24 + _tail,
    "interaction": 24 + (24 * N_BLOCKS if USE_BELIEF else 0) + _tail,
}

for name, enc in FEATURE_ENCODERS.items():
    d = feature_dim(enc)
    assert d == expected_dims[name], f"{name}: expected d={expected_dims[name]}, go

    # Every hour / day combination must produce a finite vector of constant length.
    for hour in range(24):
        for day in range(7):
            x = enc(_make_state(hour, day=day, fatigue=0.5, recency=0.5))
            assert x.shape == (d,), f"{name}: ragged output at hour={hour}"
            assert np.all(np.isfinite(x)), f"{name}: non-finite value at hour={hour

# The one-hot block must select exactly one hour.
for hour in range(24):
    assert encode_onehot(_make_state(hour))[:24].sum() == 1.0

# Hours 23 and 0 must be neighbours under the cyclic encodings but NOT under
# one-hot -- that difference is precisely what the ablation is testing.
d_cyclic = np.linalg.norm(encode_raw(_make_state(23))[:2] - encode_raw(_make_state(
d_onehot = np.linalg.norm(encode_onehot(_make_state(23))[:24] - encode_onehot(_make
print(f"distance(23:00, 00:00)  raw={d_cyclic:.3f}   onehot={d_onehot:.3f}")
```

```
# Weekend flag fires on Saturday and Sunday only.
assert encode_raw(_make_state(9, day=4))[2] == 0.0   # Friday
assert encode_raw(_make_state(9, day=5))[2] == 1.0   # Saturday

print("All feature-encoder checks passed.")
```

```
distance(23:00, 00:00)  raw=0.261   onehot=1.414
All feature-encoder checks passed.
```

In [5]:

```
# --- Demonstrating the representational limit ------------------------------
# The claim "a single sinusoid cannot fit a three-peak archetype" is easy to
# assert and easy to prove, so it is proved here rather than argued. Each basis
# is fitted by least squares to a synthetic Office-Worker-style responsiveness
# curve; the residual error is the capacity of that encoding, before any
# learning or exploration is involved.

hours = np.arange(24)

# Office Worker: receptive at the commute, at lunch, and in the evening.
def _peak(centre, width=1.6):
    return np.exp(-0.5 * (((hours - centre + 12) % 24 - 12) / width) ** 2)

target = 0.55 * _peak(8) + 0.40 * _peak(13) + 0.85 * _peak(19)

# Time-features only: the shared tail is constant here and would not change the fit
bases = {
    "raw (sin/cos)":      np.column_stack([_harmonics(h, 1) for h in hours]).T,
    "harmonic (k=1..3)":  np.column_stack([_harmonics(h, 3) for h in hours]).T,
    "onehot (24 hours)":  np.eye(24),
}

fig, ax = plt.subplots(figsize=(11, 4.2))
ax.plot(hours, target, "k-", lw=2.5, label="true click propensity", zorder=5)

for label, basis in bases.items():
    # Append an intercept column, then solve the least-squares fit.
    design = np.column_stack([basis, np.ones(24)])
    coef, *_ = np.linalg.lstsq(design, target, rcond=None)
    fit = design @ coef
    rmse = np.sqrt(np.mean((fit - target) ** 2))
    ax.plot(hours, fit, "--", lw=1.8, label=f"{label}  (RMSE {rmse:.3f})")

ax.set_xlabel("hour of day")
ax.set_ylabel("click propensity")
ax.set_title("Best achievable fit per encoding (Office Worker archetype)")
ax.set_xticks(range(0, 25, 3))
ax.legend(fontsize=9)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# Interpretation: `raw` can only place one hump, so it averages the three peaks
# into a single broad bump and mistimes every one of them. `harmonic` recovers
# the shape approximately. `onehot` fits exactly (RMSE 0) because it has one free
# parameter per hour. Any remaining LinUCB shortfall under `onehot` therefore
# cannot be attributed to the time encoding.
```



---

4. The LinUCB agent

Disjoint LinUCB (Li et al., 2010): each arm keeps its own independent linear model, with no parameters shared between arms. The proposal writes the score with arm-specific features `x_{t,a}`, which is the *hybrid* form; with three discrete actions the disjoint form is the correct and simpler choice, since every arm sees the identical context and differs only in its learned response to it.

The two equations

Each arm `a` maintains a ridge-regression estimate of expected reward from the `d × d` matrix `A_a` and the `d`-vector `b_a`:

$$\hat{\theta}_a = A_a^{-1} b_a \qquad\qquad p_a = \hat{\theta}_a^{\top} x \;+\; \alpha\sqrt{x^{\top} A_a^{-1} x}$$

The first term is the **predicted reward**. The second is the **confidence bonus** — `√(xᵀA⁻¹x)` is the estimated standard deviation of that prediction in the direction `x`, so it is large where the arm has seen few similar contexts and shrinks as evidence accumulates. Adding it implements *optimism in the face of uncertainty*: an arm is tried either because it looks good or because it is poorly understood, and either reason is a legitimate cause to gather data.

After observing reward `r`, only the **played** arm updates:

$$A_a \mathrel{+}= x x^{\top} \qquad\qquad b_a \mathrel{+}= r\,x$$

The arms that were not played received no reward signal, so touching them would fabricate evidence. This is the single easiest place to introduce a silent bug.

Implementation notes

`np.linalg.solve`, not `np.linalg.inv`. Solving `A θ = b` is both faster and numerically better conditioned than forming `A⁻¹` and multiplying. Both `θ̂` and `A⁻¹x` are

obtained from a single `solve` call by stacking `b` and `x` into one right-hand side.

`A_a` is initialised to `λI`, not zeros. With no data, `A` would otherwise be singular and uninvertible. The ridge term `λ` keeps it well conditioned and controls how wide the initial confidence interval is — larger `λ` means a more conservative start. This matters most for the `onehot` variant, where `d = 29` and each hour needs its own evidence before its estimate settles.

Ties are broken at random. At initialisation every arm has `θ̂ = 0` and an identical confidence bonus, so all three scores are exactly equal. A plain `argmax` would return index 0 — `HOLD` — every time, biasing the very first decisions toward silence purely as an artefact of tie ordering.

`greedy=True` drops the bonus. All reported evaluation runs use it, so the numbers reflect the learned policy rather than exploration noise.

No reset between episodes. `A` and `b` accumulate over the entire training run; that accumulation *is* the learning.

In [6]:
```python
class LinUCBAgent(Agent):
    """Disjoint LinUCB contextual bandit (Li et al., 2010).

    Serves as the deliberately stateless baseline in the CANE comparison. It
    can see fatigue in its context vector, but has no transition model, so it
    cannot represent the fact that its own send is what *causes* fatigue later.

    Efficiency note: A^-1 is maintained incrementally via the Sherman-Morrison
    identity rather than re-solving a d x d system at every step. Because each
    update is a rank-1 modification A <- A + x x^T, the inverse can be updated
    directly:

        (A + x x^T)^-1 = A^-1 - (A^-1 x)(x^T A^-1) / (1 + x^T A^-1 x)

    This reduces per-step cost from O(d^3) to O(d^2), which matters most for the
    one-hot encoding (d = 29) and makes the alpha sweep tractable. `self.A` is
    still maintained so the cached inverse can be checked against an exact solve.
    """

    def __init__(self, encoder_name="raw", alpha=1.0, lam=1.0,
                 n_actions=N_ACTIONS, seed=0, label=None):
        """
        Args:
            encoder_name: key into FEATURE_ENCODERS ("raw"/"harmonic"/"onehot").
            alpha: exploration coefficient. 0 = greedy ridge regression, which
                is a useful ablation for separating the value of exploration
                from the value of the linear model itself.
            lam: ridge parameter. Scales the initial A = lam * I, keeping it
                invertible before any data and setting the initial confidence
                width.
            seed: controls tie-breaking only; the agent is otherwise
                deterministic given its data.
            label: overrides the display name in results tables.
```

```
        """
        self.encoder_name = encoder_name
        self.encoder = FEATURE_ENCODERS[encoder_name]
        self.d = feature_dim(self.encoder)

        self.alpha = float(alpha)
        self.lam = float(lam)
        self.n_actions = int(n_actions)
        self._label = label
        self.rng = np.random.default_rng(seed)

        self._init_stats(self.d)

    def _init_stats(self, d):
        """Allocate per-arm sufficient statistics.

        A starts at lam*I so it is invertible with zero observations; its
        inverse is therefore I/lam.
        """
        eye = np.eye(d)
        self.A = np.stack([eye * self.lam for _ in range(self.n_actions)])
        self.A_inv = np.stack([eye / self.lam for _ in range(self.n_actions)])
        self.b = np.zeros((self.n_actions, d))
        self.n_pulls = np.zeros(self.n_actions, dtype=int)

    @property
    def name(self):
        """Distinguishes the feature variants, which are all this same class."""
        return self._label or f"LinUCB-{self.encoder_name}"

    def act(self, state, greedy=False):
        """Score every arm and play the highest.

        Score = predicted reward + alpha * (confidence width).
        Under `greedy` the confidence term is dropped, leaving pure exploitation.
        """
        x = self.encoder(state)
        scores = np.empty(self.n_actions)

        for a in range(self.n_actions):
            A_inv = self.A_inv[a]
            theta = A_inv @ self.b[a]           # ridge estimate
            mean = float(theta @ x)
            if greedy:
                scores[a] = mean
            else:
                # max(..., 0) guards against tiny negative values from
                # floating-point error; the quadratic form is >= 0 in exact
                # arithmetic because A is positive definite.
                width = np.sqrt(max(float(x @ (A_inv @ x)), 0.0))
                scores[a] = mean + self.alpha * width

        # Random tie-breaking. At t=0 all arms score identically, and a plain
        # argmax would always return HOLD, biasing the opening decisions.
        best = np.flatnonzero(scores == scores.max())
        action = int(self.rng.choice(best)) if best.size > 1 else int(best[0])
```

```
        return action, {}

    def update(self, state, action, reward, next_state=None, done=False, aux=None):
        """Rank-1 ridge update on the PLAYED arm only.

        `next_state`, `done` and `aux` are accepted to satisfy the shared Agent
        interface and are deliberately unused: LinUCB has no transition model.
        That is precisely the limitation the RQ2 comparison is designed to expose.
        """
        x = self.encoder(state)

        # Only the played arm observed a reward. Updating any other arm would
        # invent evidence that was never collected.
        self.A[action] += np.outer(x, x)
        self.b[action] += reward * x

        # Sherman-Morrison rank-1 inverse update.
        A_inv = self.A_inv[action]
        Ax = A_inv @ x
        self.A_inv[action] = A_inv - np.outer(Ax, Ax) / (1.0 + float(x @ Ax))

        self.n_pulls[action] += 1
        return {"arm": action, "reward": reward}

    def theta(self, action):
        """Current coefficient estimate for one arm (for the analysis section)."""
        return self.A_inv[action] @ self.b[action]

    def inverse_drift(self):
        """Largest discrepancy between the cached inverse and an exact solve.

        Sherman-Morrison accumulates floating-point error over many updates, so
        this is checked rather than assumed. Values around 1e-10 are expected.
        """
        return max(float(np.abs(self.A_inv[a] - np.linalg.inv(self.A[a])).max())
                   for a in range(self.n_actions))

    def __repr__(self):
        return (f"LinUCBAgent(encoder={self.encoder_name!r}, d={self.d}, "
                f"alpha={self.alpha}, lam={self.lam})")


print("LinUCBAgent defined (Sherman-Morrison incremental inverse).")
```

```
LinUCBAgent defined (Sherman-Morrison incremental inverse).
```

---

5. Correctness test on a synthetic bandit

Before running against CANE, the implementation is verified on a problem whose answer is known in advance.

Why this is necessary. On the real environment a poor LinUCB score is ambiguous: it could be the genuine limitation the study is trying to demonstrate, or it could be a bug. Those two explanations are indistinguishable from the score alone. So the agent is first run on a synthetic contextual bandit with a **planted** parameter vector `θ*` per arm and reward `r = xᵀθ*_a + noise`. Here the correct answer is known, which makes two things checkable:

1. **Parameter recovery** — `θ̂_a` must converge to `θ*_a`.
2. **Sub-linear regret** — cumulative regret against an oracle that always plays the truly best arm must flatten. A correct bandit stops paying for its mistakes; a broken one accumulates regret at a constant rate.

Passing this means any subsequent shortfall on CANE is attributable to the *bandit formulation*, not to the code. That distinction is the whole basis of the RQ2 argument, so it is worth establishing explicitly.

An `alpha = 0` run is included as a contrast: pure exploitation with no confidence bonus, which tends to lock onto whichever arm looked good early and stop gathering evidence. It isolates how much of the performance comes from exploration rather than from the linear model.

In [7]:
```python
def run_synthetic_bandit(agent, theta_star, n_steps=6000, noise_sd=0.1, seed=0):
    """Run `agent` on a linear contextual bandit with known parameters.

    Reward for arm a in context x is  x @ theta_star[a] + N(0, noise_sd).
    Because theta_star is known, the oracle's choice is computable at every
    step, which makes true regret measurable.

    Returns:
        (cumulative_regret, per-step chosen arm) as numpy arrays.
    """
    rng = np.random.default_rng(seed)
    d = theta_star.shape[1]

    regret = np.empty(n_steps)
    chosen = np.empty(n_steps, dtype=int)
    running = 0.0

    for t in range(n_steps):
        # Random context, with a constant final element acting as an intercept.
        x = rng.normal(size=d)
        x[-1] = 1.0

        action, aux = agent.act(x)
        reward = float(x @ theta_star[action] + rng.normal(0.0, noise_sd))

        # Regret is measured against expected rewards, not the noisy draw, so
        # it reflects decision quality rather than luck.
        expected = theta_star @ x
        running += float(expected.max() - expected[action])

        agent.update(x, action, reward, next_state=None, done=False, aux=aux)
```

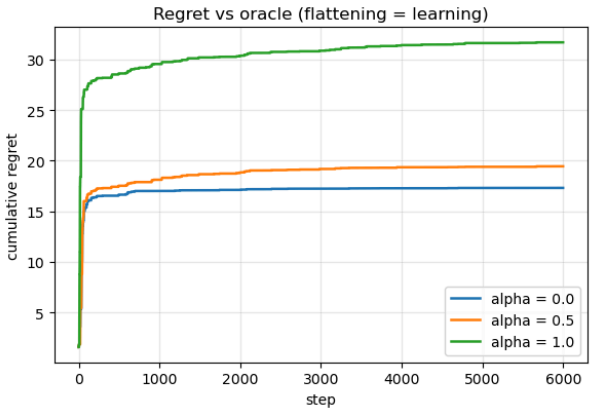
```
        regret[t] = running
        chosen[t] = action

    return regret, chosen


# --- Set up a known problem ------------------------------------------------
D_SYN = 6
N_STEPS = 6000
rng_setup = np.random.default_rng(42)
THETA_STAR = rng_setup.normal(size=(N_ACTIONS, D_SYN))

# The synthetic contexts are plain vectors, so the agent must not apply a CANE
# feature encoder here. An identity encoder is injected for the test only.
FEATURE_ENCODERS["_identity"] = lambda s: np.asarray(s, dtype=np.float64)


def make_test_agent(alpha, seed=0):
    """LinUCB configured for the synthetic problem's dimensionality."""
    ag = LinUCBAgent(encoder_name="_identity", alpha=alpha, lam=1.0, seed=seed,
                     label=f"alpha={alpha}")
    ag.d = D_SYN                # identity encoder -> d = D_SYN, not a CANE dim
    ag._init_stats(D_SYN)       # reallocate A, A_inv and b at the right size
    return ag


results = {}
for alpha in [0.0, 0.5, 1.0]:
    ag = make_test_agent(alpha)
    regret, chosen = run_synthetic_bandit(ag, THETA_STAR, n_steps=N_STEPS)
    err = np.array([np.linalg.norm(ag.theta(a) - THETA_STAR[a]) for a in range(N_AC
    results[alpha] = (regret, chosen, err, ag)
    print(f"alpha={alpha:<4}  final regret={regret[-1]:8.2f}   "
          f"mean ||theta_hat - theta*||={err.mean():.4f}   "
          f"arm pulls={ag.n_pulls.tolist()}")
```

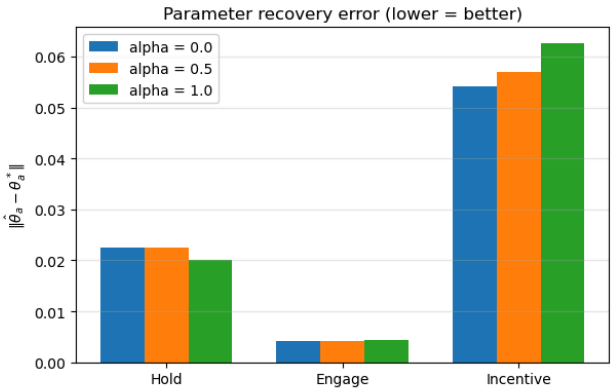
```
alpha=0.0   final regret=   17.32   mean ||theta_hat - theta*||=0.0270   arm pulls=
[1635, 3725, 640]
alpha=0.5   final regret=   19.45   mean ||theta_hat - theta*||=0.0279   arm pulls=
[1640, 3696, 664]
alpha=1.0   final regret=   31.70   mean ||theta_hat - theta*||=0.0291   arm pulls=
[1642, 3674, 684]
```

In [8]:

```
# --- Verdict ---------------------------------------------------------------
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.2))

for alpha, (regret, chosen, err, ag) in results.items():
    ax1.plot(regret, lw=1.8, label=f"alpha = {alpha}")
ax1.set_xlabel("step")
ax1.set_ylabel("cumulative regret")
ax1.set_title("Regret vs oracle (flattening = learning)")
ax1.legend()
ax1.grid(alpha=0.3)

# Parameter recovery: ||theta_hat - theta*|| per arm.
```

```python
width, offset = 0.25, -0.25
for alpha, (regret, chosen, err, ag) in results.items():
    ax2.bar(np.arange(N_ACTIONS) + offset, err, width, label=f"alpha = {alpha}")
    offset += width
ax2.set_xticks(range(N_ACTIONS))
ax2.set_xticklabels([ACTION_NAMES[a] for a in range(N_ACTIONS)])
ax2.set_ylabel(r"$\|\hat\theta_a - \theta^*_a\|$")
ax2.set_title("Parameter recovery error (lower = better)")
ax2.legend()
ax2.grid(alpha=0.3, axis="y")

plt.tight_layout()
plt.show()


# --- Automated assertions ----------------------------------------------------
for alpha, (regret, chosen, err, ag) in results.items():
    # 1. Parameters recovered to within noise.
    assert err.max() < 0.15, f"alpha={alpha}: theta not recovered ({err.max():.3f})

    # 2. Regret must be sub-linear: the second half of the run should accrue
    #    substantially less regret than the first. A constant-rate accumulation
    #    would mean the agent never stopped making the same mistake.
    half = len(regret) // 2
    first, second = regret[half], regret[-1] - regret[half]
    assert second < first, f"alpha={alpha}: regret not sub-linear ({first:.1f} -> {

    # 3. The Sherman-Morrison cached inverse must still agree with an exact
    #    solve after 6000 rank-1 updates. This is the risk that optimisation
    #    introduces, so it is measured rather than assumed.
    drift = ag.inverse_drift()
    assert drift < 1e-6, f"alpha={alpha}: inverse drift too large ({drift:.2e})"

    print(f"alpha={alpha:<4} PASS   regret 1st half={first:6.2f}  2nd half={second:
          f"max theta error={err.max():.4f}  inverse drift={drift:.2e}")

print("\nImplementation verified against a known ground truth.")

# NOTE on the alpha ordering. Here alpha = 0 attains the LOWEST regret, which
# looks backwards but is expected: the synthetic contexts are drawn i.i.d. from
# a Gaussian, so context variation alone already explores the parameter space
# and an explicit bonus buys nothing while still costing sub-optimal pulls.
# CANE is not like this -- its contexts are highly structured (the hour cycles
# deterministically, fatigue is autocorrelated), so coverage is NOT free there.
# This test therefore validates correctness only; alpha must still be tuned on
# the real environment in Section 7.
```






```
alpha=0.0  PASS   regret 1st half= 17.24  2nd half=  0.08  max theta error=0.0541  i
nverse drift=1.49e-16
alpha=0.5  PASS   regret 1st half= 19.19  2nd half=  0.26  max theta error=0.0570  i
nverse drift=1.14e-16
alpha=1.0  PASS   regret 1st half= 30.87  2nd half=  0.83  max theta error=0.0626  i
nverse drift=2.19e-16

Implementation verified against a known ground truth.
```

---

6. The CANE environment

Implements the simulator specified in the proposal: five hidden archetypes, a fatigue accumulation–recovery process, a fatigue-attenuated click model, and a fatigue-driven churn model.

> **Ownership note.** The environment is a shared team deliverable. This is a reference implementation used to develop and test the LinUCB agent against; the parameter choices below need team sign-off before any reported run.

Semantics fixed here

The proposal leaves the timestep granularity implicit (§4e's worked example uses irregular gaps, while the state carries an integer hour). This implementation fixes:

- **1 step = 1 hour**, **1 episode = 168 steps** (one week)
- Episode ends early on opt-out, which is terminal
- One archetype is drawn per episode and never revealed to the agent

Fatigue is read *before* the action's increment when computing both the reward penalty and the click probability. This follows the worked example, where a step at `F = 0.10` incurs a penalty of `2 × 0.10 = 0.20` and *then* decays.

Parameters not specified in the proposal

These are engineering choices, justified by their consequences rather than by citation. Each is exposed in `CANE_CONFIG` so the team can revise them, and so the reward-weight sensitivity study can sweep them.

Parameter	Value	Rationale
`mu` (fatigue attenuation)	0.80	at maximum fatigue the click probability is cut to 20% of baseline — severe but not absorbing
`w[ENGAGE]`	1.00	reference message type
`w[INCENTIVE]`	1.30	a promo is more compelling per send; this is what makes the action choice non-trivial
`churn_threshold`	0.70	matches the "high-fatigue zone" drawn in the proposal's Figure 1.2
`gamma_0`, `gamma_1`	−7.0, 4.0	at `F = 1.0` the hourly opt-out hazard is ≈ 4.7%, so a saturated user is near-certain to churn within a week, while a policy holding `F < 0.7` never churns at all. This reproduces the intended contrast between the Random sender and a paced policy.

The streak term — a specification conflict

The proposal defines the streak bonus **three inconsistent ways**: §4e-ii writes the formula as a cumulative sum $\Sigma_k \beta^k$, the surrounding text says each click adds the *marginal* term β^n, and the worked example applies the marginal term (+1.15, then +1.32).

The distinction is not cosmetic. Paid cumulatively, the bonus reaches ≈124 by `n = 20` — which would dwarf the −30 churn penalty and invert the reward ordering (`R_churn ≫ R_click`) that the proposal relies on for its entire justification. The agent would then learn that a long streak is worth risking opt-out for.

This implementation therefore defaults to the **marginal** term with a cap, and exposes `streak_mode` so the alternative can be measured rather than argued about. **This needs a team decision before any reported run.**

In [9]:
```
# --- Configuration -------------------------------------------------------
# Every tunable lives here so experiments can sweep them without touching the
# environment code, and so the reward-weight sensitivity study has one target.

# Relevance weight of each message type, per archetype. An Incentive Nudge is a
# discount, so it lands harder with price-sensitive users (students, deal-seekers)
# and adds little for time-poor ones who value relevance over savings.
ARCHETYPE_W = {
    "OfficeWorker":     {ENGAGE: 1.15, INCENTIVE: 1.05},
    "NightOwlStudent":  {ENGAGE: 0.90, INCENTIVE: 1.60},
    "NightShiftWorker": {ENGAGE: 1.00, INCENTIVE: 1.25},
    "NormalStudent":    {ENGAGE: 0.90, INCENTIVE: 1.55},
    "Housewife":        {ENGAGE: 1.05, INCENTIVE: 1.45},
}
```

```python
CANE_CONFIG = dict(
    # Episode structure
    steps_per_episode=168,      # 1 step = 1 hour, 1 episode = 1 week

    # Fatigue dynamics:  F' = clip(lam*F + kappa*1{a != 0}, 0, 1)
    lam=0.90,                   # recovery factor (proposal §4b)
    kappa={HOLD: 0.0, ENGAGE: 0.12, INCENTIVE: 0.18},   # Incentive is pushier

    # Click model:  P = clip(p0(arch,h,d) * (1 - mu*F) * w(a), 0, 1)
    mu=0.80,                    # fatigue attenuation strength
    w_arch=ARCHETYPE_W,

    # Churn model:  P(opt-out | F) = sigmoid(g0 + g1*F), only above threshold
    churn_threshold=0.70,
    gamma_0=-7.0,
    gamma_1=4.0,

    # Reward weights (proposal §4e). Ordering: R_churn >> R_click > W_fat > W_send
    R_click=10.0,
    W_send={HOLD: 0.0, ENGAGE: 1.0, INCENTIVE: 2.5},    # a promo has real margin c
    W_fat=2.0,
    R_churn=30.0,

    # Retention-streak shaping
    beta=1.15,
    streak_mode="marginal",     # "marginal" (beta**n) or "cumulative" (sum)
    streak_cap=5.0,             # ceiling; None disables. Protects the reward order
    streak_lapse_hours=36,      # no click for this long -> streak resets

    # In-app activity: probability the user is already in the app, which scales
    # with their current receptiveness (an engaged user is more likely present).
    active_scale=0.6,
)


# --- Archetype responsiveness curves ----------------------------------------
# Each archetype is a sum of Gaussian "receptive windows" over the 24h clock.
# Amplitudes are set so peak baseline click propensity sits around 0.35-0.50,
# which after fatigue attenuation and the message weight yields open rates in a
# plausible range rather than an implausibly generous one.

ARCHETYPES = ["OfficeWorker", "NightOwlStudent", "NightShiftWorker",
              "NormalStudent", "Housewife"]


def _bump(hours, centre, width, amp):
    """A Gaussian receptive window, wrapped correctly around midnight."""
    delta = (hours - centre + 12.0) % 24.0 - 12.0
    return amp * np.exp(-0.5 * (delta / width) ** 2)


def archetype_curve(archetype, hours=None):
    """Baseline click propensity p0 by hour for one archetype."""
    h = np.arange(24.0) if hours is None else np.asarray(hours, dtype=float)
```

```
    if archetype == "OfficeWorker":
        # Commute, lunch, evening. Suppressed through working hours.
        c = _bump(h, 7.5, 1.0, 0.42) + _bump(h, 12.5, 1.1, 0.30) + _bump(h, 19.5, 2
    elif archetype == "NightOwlStudent":
        # Peaks 22:00-02:00, quiet all morning.
        c = _bump(h, 0.0, 2.2, 0.32) + _bump(h, 22.0, 1.6, 0.26) + _bump(h, 16.0, 2
    elif archetype == "NightShiftWorker":
        # Phase-inverted: before the shift (late afternoon), after it (early am).
        c = _bump(h, 16.5, 1.6, 0.45) + _bump(h, 6.5, 1.5, 0.40) + _bump(h, 2.0, 1.
    elif archetype == "NormalStudent":
        # After school, then an evening study break. Asleep by midnight.
        c = _bump(h, 16.0, 1.6, 0.45) + _bump(h, 21.0, 1.5, 0.42) + _bump(h, 7.0, 1
    elif archetype == "Housewife":
        # The most time-flexible: broad daytime availability, family-hours dip.
        c = _bump(h, 10.0, 2.4, 0.42) + _bump(h, 14.5, 2.4, 0.40) + _bump(h, 20.5,
    else:
        raise ValueError(f"unknown archetype: {archetype}")

    return np.clip(c, 0.0, 1.0)


# Precomputed lookup: (archetype, hour) -> p0. Avoids recomputing exponentials
# 100k+ times during training.
ARCHETYPE_TABLE = {a: archetype_curve(a) for a in ARCHETYPES}

# Weekend modulation. Work- and school-bound routines loosen at the weekend;
# the Housewife and Night-Owl routines are largely unaffected.
WEEKEND_FLATTEN = {"OfficeWorker": 0.55, "NormalStudent": 0.50,
                   "NightShiftWorker": 0.30, "NightOwlStudent": 0.15,
                   "Housewife": 0.05}

print(f"{len(ARCHETYPES)} archetypes defined:", ", ".join(ARCHETYPES))
```

```
5 archetypes defined: OfficeWorker, NightOwlStudent, NightShiftWorker, NormalStuden
t, Housewife
```

In [10]:

```
# --- The simulated user -----------------------------------------------------
# The environment is the assignment's model of one app user over a week: 168
# hourly steps, each of which is a decision point. There is no external dataset;
# this class IS the data-generating process, and the archetype tables above are
# its ground truth.
#
# The agent never observes the archetype or the true receptiveness curve. It
# sees only the belief state assembled in `_observe`, so learning who this
# person is has to happen through reward alone. That is the whole problem.
class CANEEnv:
    """Context-Aware Notification Engine simulator.

    Gymnasium-style API (`reset` -> (obs, info); `step` -> (obs, reward,
    terminated, truncated, info)) but without a gymnasium dependency, so the
    LinUCB notebook runs standalone.

    NOTE for the DQN / PPO implementations: to use this with stable-baselines3
    it must subclass `gymnasium.Env` and declare
        observation_space = spaces.Box(low, high, shape=(STATE_DIM,), float32)
        action_space      = spaces.Discrete(N_ACTIONS)
```

```
    The dynamics below are unchanged by that; only the declarations are added.
    """

    def __init__(self, config=None, archetype=None, seed=0):
        """
        Args:
            config: overrides for CANE_CONFIG.
            archetype: pin a single archetype (for per-archetype evaluation).
                If None, one is drawn uniformly at each reset.
            seed: RNG seed. Evaluation uses reserved seeds so that every agent
                is scored on the identical sequence of users and random draws.
        """
        self.cfg = {**CANE_CONFIG, **(config or {})}
        self.fixed_archetype = archetype
        self.rng = np.random.default_rng(seed)
        self.reset()

    # -- helpers ---------------------------------------------------------

    def _p0(self):
        """Baseline click propensity for the current archetype, hour and day."""
        # Baseline receptiveness for this person at this hour of the day.
        # On weekends (day >= 5) it is blended toward the person's daily mean:
        # a weekday commuter's sharp 08:00 peak flattens out on a Saturday.
        base = ARCHETYPE_TABLE[self.archetype][self.hour]
        if self.day >= 5:
            # At the weekend a work-bound routine flattens toward its own mean:
            # the peaks soften and the troughs lift.
            f = WEEKEND_FLATTEN[self.archetype]
            base = (1.0 - f) * base + f * ARCHETYPE_TABLE[self.archetype].mean()
        return float(base)

    def _observe(self):
        """Assemble the observation vector. Hour is RAW; encoding is the agent's jo
        # The belief state. Note what is absent: the archetype, the true
        # receptiveness curve, and the click probability. The agent gets only
        # what an app could actually log about a user.
        s = np.zeros(STATE_DIM, dtype=np.float32)
        s[IDX_HOUR] = self.hour
        s[IDX_DAY] = self.day
        s[IDX_ACTIVE] = self.active
        s[IDX_FATIGUE] = self.fatigue
        s[IDX_RECENCY] = self.recency

        # Beta-prior smoothing. Early in an episode a block may have zero sends,
        # and a raw clicks/sends ratio would be 0/0. Adding the prior makes an
        # unobserved block read as 'unknown' rather than as 'known to be dead'.
        a0, b0 = BELIEF_PRIOR
        s[IDX_ACT_RATE:IDX_ACT_RATE + N_BLOCKS] = (
            (self.blk_act_hits + a0) / (self.blk_act_obs + a0 + b0))
        s[IDX_CLICK_RATE:IDX_CLICK_RATE + N_BLOCKS] = (
            (self.blk_clicks + a0) / (self.blk_sends + a0 + b0))
        s[IDX_TYPE_CR] = (self.type_clicks[ENGAGE] + a0) / (self.type_sends[ENGAGE]
        s[IDX_TYPE_CR + 1] = (self.type_clicks[INCENTIVE] + a0) / (self.type_sends[
        s[IDX_SINCE_CLICK] = min(self.hours_since_click / 48.0, 1.0)
        return s
```

```python
    def _roll_activity(self):
        """Whether the user is already in-app; more likely when receptive.

        Matters because the CTR convention counts a click only when the user was
        inactive: sending to someone already using the app is treated as wasted.
        """
        # Whether the user is already in the app this hour. Rolled every step,
        # including on Hold, because the app observes app-opens whether or not it
        # sent anything -- that is the free signal the belief state accumulates.
        p = min(1.0, self.cfg["active_scale"] * self._p0())
        self.active = int(self.rng.random() < p)

        b = self.hour // BLOCK_HOURS
        self.blk_act_obs[b] += 1
        self.blk_act_hits[b] += self.active

    # -- API ------------------------------------------------------------------

    def reset(self, seed=None):
        if seed is not None:
            self.rng = np.random.default_rng(seed)

        self.archetype = (self.fixed_archetype
                          or ARCHETYPES[self.rng.integers(len(ARCHETYPES))])

        self.t = 0
        self.hour = int(self.rng.integers(24))       # random start time of week
        self.day = int(self.rng.integers(7))
        self.fatigue = 0.0
        self.recency = 1.0                           # no send yet -> maximally sta
        self.hours_since_send = 24
        self.streak = 0
        self.hours_since_click = 0
        self.opted_out = False

        self.blk_act_obs = np.zeros(N_BLOCKS)
        self.blk_act_hits = np.zeros(N_BLOCKS)
        self.blk_sends = np.zeros(N_BLOCKS)
        self.blk_clicks = np.zeros(N_BLOCKS)
        self.type_sends = np.zeros(N_ACTIONS)
        self.type_clicks = np.zeros(N_ACTIONS)

        self._roll_activity()
        return self._observe(), {"archetype": self.archetype}

    def step(self, action):
        cfg = self.cfg
        action = int(action)
        sent = action != HOLD

        # Fatigue is read BEFORE this action's increment, for both the reward
        # penalty and the click probability (matches the proposal's worked example)
        f_now = self.fatigue

        # --- 1. Click / response model ---------------------------------------
```

```
        # A click requires a send AND an inactive user (CTR convention: the
        # metric is winning back an absent user, not pinging a present one).
        clicked = False
        # A click is only possible on a send, and only when the user is NOT
        # already active in the app: a notification delivered to someone already
        # using the app earns nothing. Click probability then scales the hour's
        # receptiveness down by current fatigue and by the per-archetype weight
        # of this message type.
        if sent and not self.active:
            w_a = cfg["w_arch"][self.archetype][action]
            p_click = np.clip(self._p0() * (1.0 - cfg["mu"] * f_now) * w_a, 0.0, 1.
            clicked = bool(self.rng.random() < p_click)

        blk = self.hour // BLOCK_HOURS
        if sent:
            self.blk_sends[blk] += 1
            self.blk_clicks[blk] += clicked
            self.type_sends[action] += 1
            self.type_clicks[action] += clicked

        # --- 2. Retention-streak shaping ----------------------------------
        if clicked:
            self.streak += 1
            self.hours_since_click = 0
        else:
            self.hours_since_click += 1
            if self.hours_since_click > cfg["streak_lapse_hours"]:
                self.streak = 0        # lapsed: the bonus collapses to zero

        # Consecutive clicks compound, which is what makes this a sequential
        # problem rather than a per-hour one: the value of a click depends on
        # whether the previous ones landed.
        streak_bonus = 0.0
        if clicked and self.streak > 0:
            if cfg["streak_mode"] == "cumulative":
                b = cfg["beta"]
                streak_bonus = sum(b ** k for k in range(1, self.streak + 1))
            else:                                   # "marginal"
                streak_bonus = cfg["beta"] ** self.streak
            if cfg["streak_cap"] is not None:
                streak_bonus = min(streak_bonus, cfg["streak_cap"])

        # --- 3. Fatigue dynamics ------------------------------------------
        self.fatigue = float(np.clip(cfg["lam"] * f_now + cfg["kappa"][action], 0.0

        # --- 4. Churn model (terminal) ------------------------------------
        if self.fatigue > cfg["churn_threshold"]:
            hazard = 1.0 / (1.0 + np.exp(-(cfg["gamma_0"] + cfg["gamma_1"] * self.f
            self.opted_out = bool(self.rng.random() < hazard)

        # --- 5. Reward ----------------------------------------------------
        reward = (cfg["R_click"] * clicked
                  - cfg["W_send"][action]
                  - cfg["W_fat"] * f_now
                  - cfg["R_churn"] * self.opted_out
                  + streak_bonus)
```

```
        # --- 6. Advance time ------------------------------------------------
        self.t += 1
        self.hour = (self.hour + 1) % 24
        if self.hour == 0:
            self.day = (self.day + 1) % 7

        # Recency, capped at 24h and normalised, is the agent's only direct view
        # of how recently it last made contact.
        self.hours_since_send = 0 if sent else self.hours_since_send + 1
        self.recency = float(min(self.hours_since_send / 24.0, 1.0))
        self._roll_activity()

        terminated = self.opted_out
        truncated = self.t >= cfg["steps_per_episode"]

        info = {
            "archetype": self.archetype,
            "clicked": clicked, "sent": sent,
            "fatigue": self.fatigue, "fatigue_pre": f_now,
            "opted_out": self.opted_out, "streak": self.streak,
            "streak_bonus": streak_bonus,
            "hour": int((self.hour - 1) % 24), "active_pre": self.active,
        }
        return self._observe(), float(reward), terminated, truncated, info


print("CANEEnv defined.")
```

```
CANEEnv defined.
```

In [11]:

```
# --- Non-learning baselines -----------------------------------------------
# These answer RQ1: does learning beat not learning? Both implement the same
# Agent interface so the harness treats them identically to a trained agent.

# --- Non-learning baselines -----------------------------------------------
# Fixed-18:00 is the industry default this project argues against: one
# hard-coded send time for everyone. Random is the lower bound. Neither
# learns, so both are evaluated once rather than across training seeds.
class FixedScheduleAgent(Agent):
    """Sends one Engagement Nudge per day at a fixed hour (default 18:00).

    Represents the conventional context-blind notification system: the same
    push, to every user, at the same time, regardless of state.
    """

    def __init__(self, hour=18, action=ENGAGE):
        self.hour, self.action = hour, action

    @property
    def name(self):
        return f"Fixed-{self.hour:02d}:00"

    def act(self, state, greedy=False):
        return (self.action if int(state[IDX_HOUR]) == self.hour else HOLD), {}
```

```
    def update(self, *args, **kwargs):
        return {}


class RandomAgent(Agent):
    """Uniformly random action. The floor: what happens with no policy at all."""

    def __init__(self, seed=0):
        self.rng = np.random.default_rng(seed)

    @property
    def name(self):
        return "Random"

    def act(self, state, greedy=False):
        return int(self.rng.integers(N_ACTIONS)), {}

    def update(self, *args, **kwargs):
        return {}


# --- Harness ----------------------------------------------------------------

# --- The evaluation harness -------------------------------------------
# One function drives both training and evaluation, which is what keeps
# them honest: `learn=True, greedy=False` trains, `learn=False,
# greedy=True` scores a frozen policy. Passing explicit `seeds` replays
# an identical set of episodes for every agent.
def run_episodes(agent, env, n_episodes=None, seeds=None, learn=True,
                 greedy=False, collect_log=False):
    """Run `agent` in `env` and return aggregate metrics.

    Args:
        n_episodes: number of episodes (ignored if `seeds` is given).
        seeds: explicit per-episode reset seeds. This is the evaluation
            protocol: passing the same seed list to every agent guarantees they
            all face the identical sequence of archetypes and starting
            conditions, so differences in score reflect the policy rather than
            luck of the draw. (Within an episode the RNG streams necessarily
            diverge, because the agents take different actions.)
        learn: call agent.update(). False for evaluation and for baselines.
        greedy: act without exploration. Used for all reported results.
        collect_log: also return a per-step record, for the decision-distribution
            plots and the animation.

    Returns:
        (metrics, log, per-episode rewards, hour x action decision counts)
    """
    if seeds is None:
        seeds = [None] * int(n_episodes)

    ep_rewards, ep_sends, ep_clicks, ep_optouts, ep_lengths = [], [], [], [], []
    # Counts actions by clock hour, which is what the policy heatmaps and the
    # personalisation analysis are built from later.
    hour_actions = np.zeros((24, N_ACTIONS), dtype=int)
    log = []
```

```
    for ep, sd in enumerate(seeds):
        state, info = env.reset(seed=sd)
        agent.reset()
        total_r = sends = clicks = 0.0

        while True:
            action, aux = agent.act(state, greedy=greedy)
            next_state, reward, terminated, truncated, info = env.step(action)

            if learn:
                agent.update(state, action, reward, next_state, terminated, aux)

            total_r += reward
            sends += info["sent"]
            clicks += info["clicked"]
            hour_actions[info["hour"], action] += 1

            if collect_log:
                log.append({"episode": ep, "step": env.t, "hour": info["hour"],
                            "archetype": info["archetype"], "action": action,
                            "clicked": info["clicked"], "sent": info["sent"],
                            "fatigue": info["fatigue"], "reward": reward,
                            "streak": info["streak"], "opted_out": info["opted_out"

            state = next_state
            if terminated or truncated:
                ep_optouts.append(float(terminated))
                break

        ep_rewards.append(total_r)
        ep_sends.append(sends)
        ep_clicks.append(clicks)
        ep_lengths.append(env.t)

    # CTR is clicks per SEND, not per step, and is defined as 0.0 when the
    # agent never sent. A never-send policy therefore reports ctr 0.00 and
    # reward 0.00 -- neither is a missing value; both are what silence earns.
    total_sends = float(np.sum(ep_sends))
    metrics = {
        "agent": agent.name,
        "reward_mean": float(np.mean(ep_rewards)),
        "reward_std": float(np.std(ep_rewards)),
        # CTR = clicks / sends (clicks counted only when the user was inactive).
        "ctr": float(np.sum(ep_clicks) / total_sends) if total_sends > 0 else 0.0,
        "sends_per_episode": float(np.mean(ep_sends)),
        "clicks_per_episode": float(np.mean(ep_clicks)),
        "optout_rate": float(np.mean(ep_optouts)),
        "episode_length": float(np.mean(ep_lengths)),
    }
    return metrics, (log if collect_log else None), np.array(ep_rewards), hour_acti


# --- Evaluation protocol -----------------------------------------------------
# Reserved seeds, disjoint from anything used in training. This is the RL
# analogue of a held-out test set: the same episodes score every method.
```

```
# QUICK trades statistical strength for speed while iterating; every reported
# number should come from a QUICK = False run.

# --- The shared evaluation protocol -------------------------------------
# These four values are identical in all four notebooks. That identity is
# what makes the cross-algorithm comparison valid, and `check_results.py`
# verifies it afterwards by confirming the deterministic baseline rows
# agree exactly across the four result files.
#
# The 900,000+ evaluation seeds are disjoint from the 1000+ training and
# 7000+ evaluation environment seeds below, so nothing is scored on an
# episode it was trained on. QUICK is a smoke-test switch: its numbers
# are NOT reportable.
QUICK = False

EVAL_SEEDS = list(range(900_000, 900_050 if QUICK else 900_200))
TRAIN_EPISODES = 150 if QUICK else 600
N_SEEDS = 2 if QUICK else 5

print(f"Eval protocol: {len(EVAL_SEEDS)} reserved episodes; "
      f"training budget {TRAIN_EPISODES} episodes "
      f"(~{TRAIN_EPISODES * CANE_CONFIG['steps_per_episode']:,} steps); "
      f"{N_SEEDS} seeds." + ("   [QUICK]" if QUICK else ""))
```

```
Eval protocol: 200 reserved episodes; training budget 600 episodes (~100,800 steps);
5 seeds.
```

In [ ]:

```
# --- Training / evaluation harness --------------------------------------

# Probe episodes for mid-training snapshots. Disjoint from the training seeds
# (1000+) and from EVAL_SEEDS, so watching a policy learn never contaminates a
# reported number.
PROBE_SEEDS = list(range(950_000, 950_020))
SNAPSHOT_ARCHETYPES = ("OfficeWorker", "NightOwlStudent")


def policy_snapshot(agent, archetypes=SNAPSHOT_ARCHETYPES, seeds=PROBE_SEEDS):
    """Freeze the current policy and record what it does, per archetype.

    Greedy with learning disabled, so the agent is measured rather than
    advanced. `run_episodes` calls `agent.reset()` on each probe episode, so an
    agent must not hold un-consumed learning state across `reset()`.
    """
    out = {}
    for arch in archetypes:
        env = CANEEnv(seed=8000, archetype=arch)
        m, _, _, hours = run_episodes(agent, env, seeds=seeds,
                                      learn=False, greedy=True)
        out[arch] = {"hour_actions": hours, "reward": m["reward_mean"],
                     "ctr": m["ctr"], "sends": m["sends_per_episode"],
                     "optout": m["optout_rate"]}
    return out


# Trains a fresh agent per seed and scores each on the held-out episodes.
# A fresh agent per seed, not one agent re-evaluated: the spread being
```

```python
# reported is the variance of the LEARNING process, not of the scoring.
def train_and_evaluate(make_agent, n_seeds=None, train_episodes=TRAIN_EPISODES,
                       archetype=None, collect_hours=False,
                       snapshot_every=None, snapshot_seed=0,
                       snapshot_archetypes=SNAPSHOT_ARCHETYPES):
    """Train and greedily evaluate one configuration across several seeds.

    Seed averaging is this study's analogue of cross-validation: it separates a
    genuine effect from a lucky initialisation. Training and evaluation use
    disjoint seed ranges so nothing is scored on episodes it was fitted to.

    snapshot_every: pause every N training episodes and record the policy. Only
        `snapshot_seed` is instrumented -- each snapshot costs
        len(PROBE_SEEDS) x len(snapshot_archetypes) episodes, so instrumenting
        every seed would cost more than the training itself.
    """
    n_seeds = N_SEEDS if n_seeds is None else n_seeds
    per_seed, hour_stack, snapshots = [], [], []

    for seed in range(n_seeds):
        agent = make_agent(seed)
        # Distinct environment seeds for training and evaluation, so a policy
        # cannot succeed by memorising particular episodes.
        train_env = CANEEnv(seed=1000 + seed, archetype=archetype)

        if snapshot_every and seed == snapshot_seed:
            snapshots.append((0, policy_snapshot(agent, snapshot_archetypes)))
            done = 0
            while done < train_episodes:
                chunk = min(snapshot_every, train_episodes - done)
                run_episodes(agent, train_env, n_episodes=chunk,
                             learn=True, greedy=False)
                done += chunk
                snapshots.append((done, policy_snapshot(agent, snapshot_archetypes)
        else:
            run_episodes(agent, train_env, n_episodes=train_episodes,
                         learn=True, greedy=False)

        # Evaluate (greedy, frozen) on the reserved held-out episodes.
        eval_env = CANEEnv(seed=7000 + seed, archetype=archetype)
        m, _, rewards, hours = run_episodes(agent, eval_env, seeds=EVAL_SEEDS,
                                            learn=False, greedy=True)
        per_seed.append(m)
        hour_stack.append(hours)

    rewards = np.array([m["reward_mean"] for m in per_seed])
    summary = {
        "agent": per_seed[0]["agent"],
        "reward_mean": rewards.mean(),
        "reward_std": rewards.std(ddof=0),
        # ddof=1: the sample standard deviation across seeds, which is what the
        # confidence intervals in the figures are derived from.
        "reward_sem_std": rewards.std(ddof=1) if n_seeds > 1 else 0.0,
        "ctr": np.mean([m["ctr"] for m in per_seed]),
        "sends_per_episode": np.mean([m["sends_per_episode"] for m in per_seed]),
        "optout_rate": np.mean([m["optout_rate"] for m in per_seed]),
```

```
        "per_seed_rewards": rewards,
    }
    if collect_hours:
        summary["hour_actions"] = np.sum(hour_stack, axis=0)
    if snapshots:
        summary["snapshots"] = snapshots
    return summary


# Baselines have nothing to train, so they are evaluated once. Both are
# deterministic given the seed list, which is precisely why their rows
# can be used to prove the four notebooks share one protocol.
def evaluate_baseline(make_agent, archetype=None):
    """Baselines do not learn, so they are only evaluated."""
    agent = make_agent(0)
    env = CANEEnv(seed=7000, archetype=archetype)
    m, _, rewards, hours = run_episodes(agent, env, seeds=EVAL_SEEDS,
                                        learn=False, greedy=True)
    return {"agent": m["agent"], "reward_mean": m["reward_mean"],
            "reward_std": m["reward_std"], "reward_sem_std": 0.0,
            "ctr": m["ctr"], "sends_per_episode": m["sends_per_episode"],
            "optout_rate": m["optout_rate"],
            "per_seed_rewards": np.array([m["reward_mean"]]),
            "hour_actions": hours}
```

In [13]:

```
# --- Environment sanity checks ------------------------------------------
# Verify the dynamics behave as specified BEFORE drawing conclusions from any
# agent's score. A miscalibrated environment would otherwise be indistinguishable
# from a badly-performing agent.

env = CANEEnv(seed=0)

# 1. Fatigue recursion matches F' = clip(0.9F + 0.15, 0, 1) under repeated sends.
e = CANEEnv(archetype="Housewife", seed=1)
e.reset(seed=1)
f_trace, expected = [], 0.0
for _ in range(30):
    _, _, term, trunc, info = e.step(ENGAGE)
    expected = min(0.9 * expected + CANE_CONFIG["kappa"][ENGAGE], 1.0)
    f_trace.append((info["fatigue"], expected))
    if term or trunc:
        break
assert all(abs(a - b) < 1e-9 for a, b in f_trace), "fatigue recursion mismatch"
print(f"Fatigue after {len(f_trace)} consecutive sends: {f_trace[-1][0]:.4f} "
      f"(saturates toward kappa/(1-lam) = "
      f"{CANE_CONFIG['kappa'][ENGAGE]/(1-CANE_CONFIG['lam']):.2f}, clipped at 1.0)")

# 2. Holding lets fatigue decay geometrically.
e.reset(seed=2)
for _ in range(5):
    e.step(ENGAGE)
f_high = e.fatigue
for _ in range(10):
    e.step(HOLD)
print(f"Fatigue after 5 sends: {f_high:.3f} -> after 10 holds: {e.fatigue:.3f}")
assert e.fatigue < f_high, "holding must reduce fatigue"
```

```
# 3. A never-sending policy must never trigger opt-out.
class _AlwaysHold(Agent):
    @property
    def name(self): return "AlwaysHold"
    def act(self, s, greedy=False): return HOLD, {}
    def update(self, *a, **k): return {}

m_hold, _, _, _ = run_episodes(_AlwaysHold(), CANEEnv(seed=3), 30, learn=False, gre
assert m_hold["optout_rate"] == 0.0, "holding forever should never cause churn"
print(f"AlwaysHold: reward={m_hold['reward_mean']:.2f}, optout={m_hold['optout_rate
      f"(pays only the fatigue-free baseline, earns nothing)")

# 4. Spamming every hour must drive fatigue to saturation and cause churn.
class _AlwaysSend(Agent):
    @property
    def name(self): return "AlwaysSend"
    def act(self, s, greedy=False): return ENGAGE, {}
    def update(self, *a, **k): return {}

m_spam, _, _, _ = run_episodes(_AlwaysSend(), CANEEnv(seed=4), 30, learn=False, gre
print(f"AlwaysSend: reward={m_spam['reward_mean']:.2f}, optout={m_spam['optout_rate
      f"mean episode length={m_spam['episode_length']:.0f}h of 168")
assert m_spam["optout_rate"] > 0.8, "over-messaging should reliably cause churn"
assert m_spam["reward_mean"] < m_hold["reward_mean"], "spamming must be worse than

print("\nEnvironment sanity checks passed.")
```

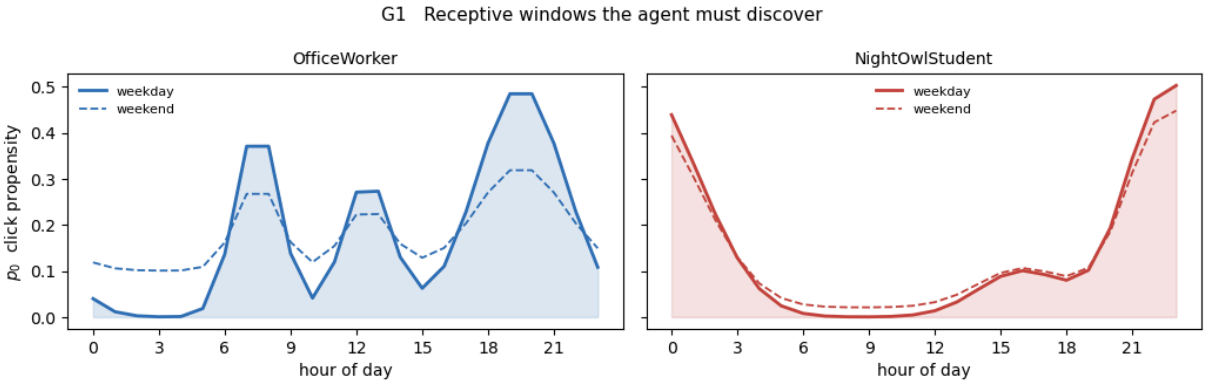
```
Fatigue after 15 consecutive sends: 0.9529 (saturates toward kappa/(1-lam) = 1.20, c
lipped at 1.0)
Fatigue after 5 sends: 0.491 -> after 10 holds: 0.171
AlwaysHold: reward=0.00, optout=0%  (pays only the fatigue-free baseline, earns noth
ing)
AlwaysSend: reward=-86.58, optout=100%, mean episode length=30h of 168

Environment sanity checks passed.
```

7. The two users

Everything below narrows to two archetypes with near-opposite routines — `OfficeWorker` and `NightOwlStudent`. Neither label appears in the state vector, so the agent has to separate them from reward alone.

In [14]:
```
# --- 7.1 Receptive windows ------------------------------------------------
from pathlib import Path

FOCUS = ["OfficeWorker", "NightOwlStudent"]
ARCH_COLOUR = {"OfficeWorker": "#2f6fb5", "NightOwlStudent": "#c4453c"}

FIGDIR = Path("figures")
FIGDIR.mkdir(exist_ok=True)

def save(fig, tag):
    fig.savefig(FIGDIR / f"{tag}.png", dpi=150, bbox_inches="tight")
```

```
def weekend_curve(archetype):
    base = ARCHETYPE_TABLE[archetype]
    f = WEEKEND_FLATTEN[archetype]
    return (1.0 - f) * base + f * base.mean()

hours = np.arange(24)
fig, axes = plt.subplots(1, 2, figsize=(10.5, 3.4), sharey=True)
for ax, arch in zip(axes, FOCUS):
    ax.fill_between(hours, ARCHETYPE_TABLE[arch], color=ARCH_COLOUR[arch], alpha=0.
    ax.plot(hours, ARCHETYPE_TABLE[arch], color=ARCH_COLOUR[arch], lw=2.0, label="w
    ax.plot(hours, weekend_curve(arch), color=ARCH_COLOUR[arch], lw=1.3, ls="--", l
    ax.set_title(arch, fontsize=10)
    ax.set_xlabel("hour of day")
    ax.set_xticks(range(0, 24, 3))
    ax.legend(frameon=False, fontsize=8)
axes[0].set_ylabel(r"$p_0$  click propensity")
fig.suptitle("G1   Receptive windows the agent must discover", fontsize=11)
fig.tight_layout()
save(fig, "G1_archetype_curves")
plt.show()

for arch in FOCUS:
    peak = int(np.argmax(ARCHETYPE_TABLE[arch]))
    print(f"{arch:16} peak {peak:02d}:00  p0={ARCHETYPE_TABLE[arch][peak]:.2f}    "
          f"dead hours (p0<0.05): {int((ARCHETYPE_TABLE[arch] < 0.05).sum())}")
```

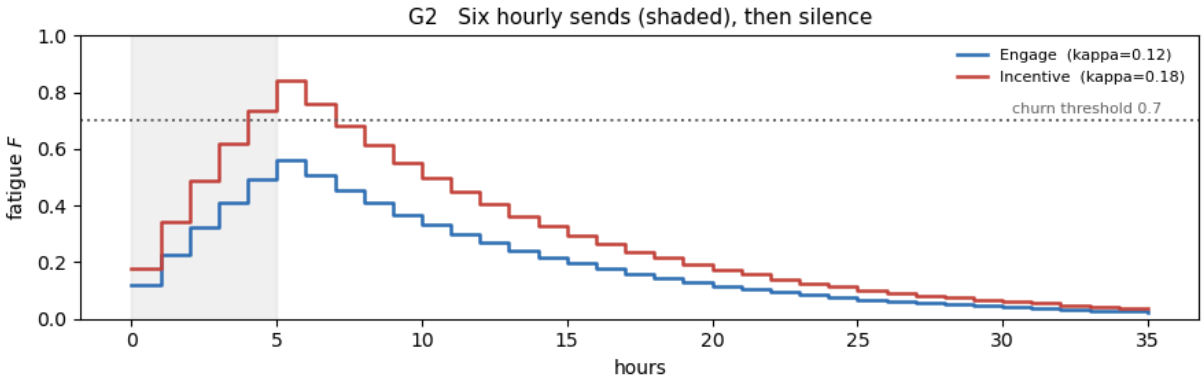



```
OfficeWorker     peak 19:00  p0=0.48   dead hours (p0<0.05): 7
NightOwlStudent  peak 23:00  p0=0.50   dead hours (p0<0.05): 9
```

In [15]:
```
# --- 7.2 Fatigue is fast to build and slow to shed --------------------------

lam = CANE_CONFIG["lam"]
kap = CANE_CONFIG["kappa"]
thr = CANE_CONFIG["churn_threshold"]

pattern = np.array([1] * 6 + [0] * 30)
fig, ax = plt.subplots(figsize=(9, 3.0))
ax.fill_between(range(len(pattern)), 0, 1, where=pattern.astype(bool),
                color="#999999", alpha=0.12, step="post")
for act, col in [(ENGAGE, "#2f6fb5"), (INCENTIVE, "#c4453c")]:
    F, trace = 0.0, []
    for a in pattern:
        F = float(np.clip(lam * F + (kap[act] if a else 0.0), 0.0, 1.0))
```

```
        trace.append(F)
    ax.step(range(len(trace)), trace, where="post", lw=1.8, color=col,
            label=f"{ACTION_NAMES[act]}  (kappa={kap[act]})")
    below = np.where(np.array(trace[6:]) < trace[5] / 2)[0]
    print(f"{ACTION_NAMES[act]:9} 6 sends -> F={trace[5]:.3f}; "
          f"{int(below[0]) + 1}h of silence to halve it")
ax.axhline(thr, color="#666666", ls=":", lw=1.4)
ax.text(len(pattern) - 0.5, thr + 0.015, f"churn threshold {thr}",
        ha="right", va="bottom", fontsize=8, color="#666666")
ax.set_xlabel("hours")
ax.set_ylabel("fatigue $F$")
ax.set_ylim(0, 1)
ax.legend(frameon=False, fontsize=8, loc="upper right")
ax.set_title("G2   Six hourly sends (shaded), then silence", fontsize=11)
fig.tight_layout()
save(fig, "G2_fatigue_dynamics")
plt.show()
```

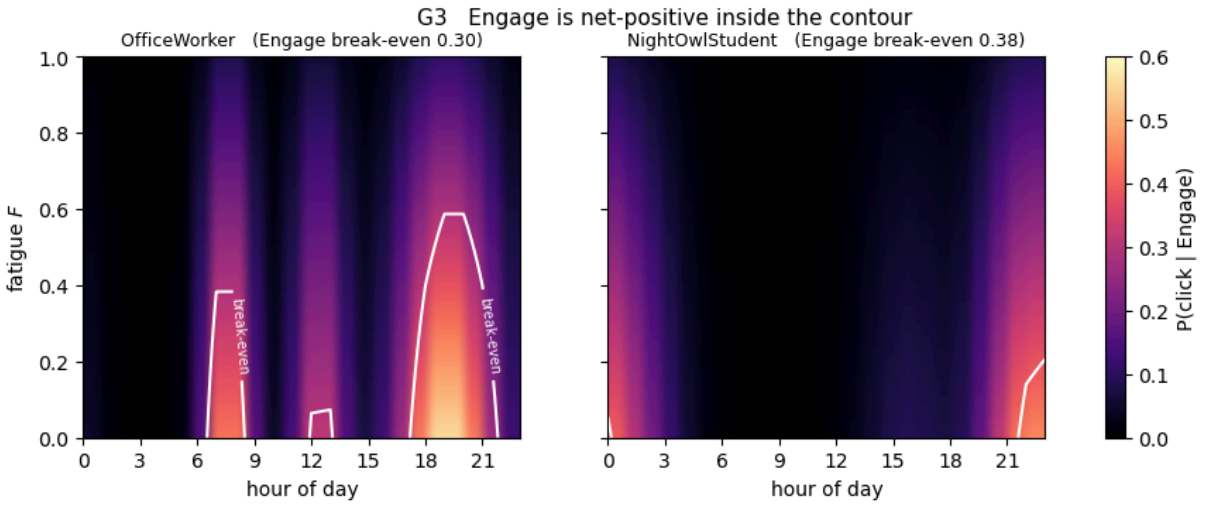
```
Engage    6 sends -> F=0.562; 7h of silence to halve it
Incentive 6 sends -> F=0.843; 7h of silence to halve it
```



In [16]:

```
# --- 7.3 Where a send is worth making ---------------------------------------

mu = CANE_CONFIG["mu"]

def break_even(archetype, action):
    cfg = CANE_CONFIG
    debt = cfg["W_fat"] * cfg["kappa"][action] / (1.0 - cfg["lam"])
    return (debt + cfg["W_send"][action]) / (cfg["R_click"] * cfg["w_arch"][archety

F_grid = np.linspace(0.0, 1.0, 101)
fig, axes = plt.subplots(1, 2, figsize=(11, 3.5), sharey=True)
for ax, arch in zip(axes, FOCUS):
    P = np.clip(np.outer(1.0 - mu * F_grid, ARCHETYPE_TABLE[arch])
                * CANE_CONFIG["w_arch"][arch][ENGAGE], 0.0, 1.0)
    im = ax.imshow(P, origin="lower", aspect="auto", cmap="magma",
                   extent=[0, 23, 0, 1], vmin=0, vmax=0.6)
    cs = ax.contour(np.arange(24), F_grid, P, levels=[break_even(arch, ENGAGE)],
                    colors="w", linewidths=1.5)
    ax.clabel(cs, fmt={break_even(arch, ENGAGE): "break-even"}, fontsize=7)
    ax.set_title(f"{arch}   (Engage break-even {break_even(arch, ENGAGE):.2f})", fo
    ax.set_xlabel("hour of day")
    ax.set_xticks(range(0, 24, 3))
axes[0].set_ylabel("fatigue $F$")
```

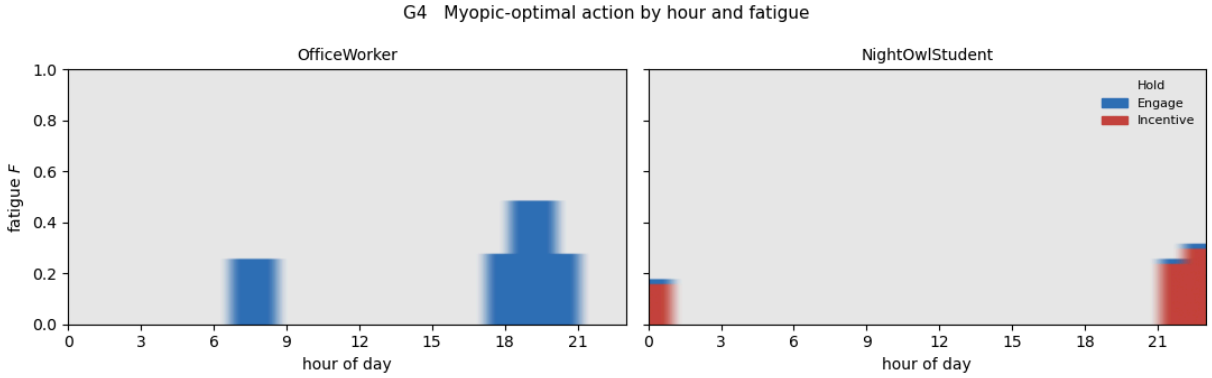
```
fig.colorbar(im, ax=axes, label="P(click | Engage)")
fig.suptitle("G3   Engage is net-positive inside the contour", fontsize=11)
save(fig, "G3_click_surface")
plt.show()
```



In [17]:

```
# --- 7.4 The policy the agent has to discover -------------------------------
# Myopic-optimal: it ignores churn risk, streak shaping and future state, so it
# is a reference for comparison rather than the true optimum.

def net_value(archetype, action, hour, F):
    cfg = CANE_CONFIG
    p = min(ARCHETYPE_TABLE[archetype][hour] * (1.0 - cfg["mu"] * F)
            * cfg["w_arch"][archetype][action], 1.0)
    debt = cfg["W_fat"] * cfg["kappa"][action] / (1.0 - cfg["lam"])
    return p * cfg["R_click"] - cfg["W_send"][action] - debt

F_GRID = np.linspace(0.0, 1.0, 101)

GROUND_TRUTH = {}
for arch in ARCHETYPES:
    M = np.zeros((len(F_GRID), 24), dtype=int)
    for j, F in enumerate(F_GRID):
        for h in range(24):
            v = {a: net_value(arch, a, h, F) for a in (ENGAGE, INCENTIVE)}
            best = max(v, key=v.get)
            M[j, h] = best if v[best] > 0 else HOLD
    GROUND_TRUTH[arch] = M

from matplotlib.colors import ListedColormap
ACTION_COLOUR = ["#e8e8e8", "#2f6fb5", "#c4453c"]

fig, axes = plt.subplots(1, 2, figsize=(11, 3.5), sharey=True)
for ax, arch in zip(axes, FOCUS):
    ax.imshow(GROUND_TRUTH[arch], origin="lower", aspect="auto",
              cmap=ListedColormap(ACTION_COLOUR), vmin=0, vmax=2, extent=[0, 23, 0,
    ax.set_title(arch, fontsize=10)
    ax.set_xlabel("hour of day")
    ax.set_xticks(range(0, 24, 3))
axes[0].set_ylabel("fatigue $F$")
axes[1].legend([plt.Rectangle((0, 0), 1, 1, color=col) for col in ACTION_COLOUR],
```

```
                ["Hold", "Engage", "Incentive"], frameon=False, fontsize=8, loc="upp
fig.suptitle("G4   Myopic-optimal action by hour and fatigue", fontsize=11)
fig.tight_layout()
save(fig, "G4_ground_truth_policy")
plt.show()

print("optimal action at F = 0        hour 000000000011111111112222")
print("                                    012345678901234567890123")
for arch in ARCHETYPES:
    print(f"{arch:18}              " + "".join({0: '.', 1: 'e', 2: 'I'}[int(a)]
                                              for a in GROUND_TRUTH[arch][0]))
```




```
optimal action at F = 0        hour 000000000011111111112222
                                    012345678901234567890123
OfficeWorker                    .......ee.........eeee..
NightOwlStudent                 I.....................II
NightShiftWorker                ......ee........ee......
NormalStudent                   ..............Ie...I..
Housewife                       .........eeeeeeee.......
```

8. Agent registry

Agents are registered once here; every experiment and figure below iterates over the registry. Adding `DQN`, `DoubleDQN` or `PPO` is a one-line `register(...)` call and requires no change to any downstream cell.

In [18]:
```
# --- 8.1 Registry ------------------------------------------------------------

AGENTS = {}

def register(name, factory, kind="learning"):
    """factory(seed) -> Agent. kind='baseline' skips training."""
    if name in AGENTS:
        raise ValueError(f"{name} already registered")
    AGENTS[name] = {"factory": factory, "kind": kind}
    return name

register("Fixed-18:00", lambda s: FixedScheduleAgent(hour=18), kind="baseline")
register("Random",      lambda s: RandomAgent(seed=s),         kind="baseline")
register("LinUCB",      lambda s: LinUCBAgent(encoder_name="onehot", alpha=0.5, see

# Teammates append here:
```

```
# register("DQN",       lambda s: DQNAgent(seed=s))
# register("DoubleDQN", lambda s: DoubleDQNAgent(seed=s))
# register("PPO",       lambda s: PPOAgent(seed=s))

print(f"{len(AGENTS)} registered:", ", ".join(AGENTS))
```

```
3 registered: Fixed-18:00, Random, LinUCB
```

In [19]:

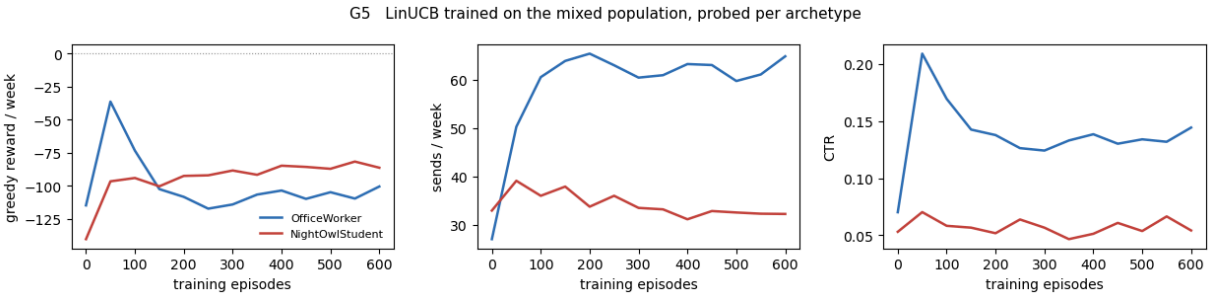
```
# --- 8.2 Driver ------------------------------------------------------------

RESULTS = {}

def run_all(archetype, names=None, n_seeds=None, train_episodes=TRAIN_EPISODES):
    out = {}
    for name in (names or list(AGENTS)):
        spec = AGENTS[name]
        if spec["kind"] == "baseline":
            r = evaluate_baseline(spec["factory"], archetype=archetype)
        else:
            r = train_and_evaluate(spec["factory"], n_seeds=n_seeds,
                                   train_episodes=train_episodes,
                                   archetype=archetype, collect_hours=True)
        r["agent"] = name
        out[name] = r
        print(f"  {archetype:16} {name:12} reward {r['reward_mean']:8.2f}  "
              f"ctr {r['ctr']:.3f}  sends {r['sends_per_episode']:6.2f}  "
              f"optout {r['optout_rate']:.3f}")
    return out


def results_table(results):
    cols = ["agent", "reward_mean", "reward_sem_std", "ctr",
            "sends_per_episode", "optout_rate"]
    df = pd.DataFrame([{c: r[c] for c in cols} for r in results.values()])
    return df.sort_values("reward_mean", ascending=False).reset_index(drop=True)


for arch in FOCUS:
    RESULTS[arch] = run_all(arch)

for arch in FOCUS:
    print(f"\n{arch}")
    print(results_table(RESULTS[arch]).to_string(index=False,
                                                 float_format=lambda v: f"{v:8.3f}"
```

```
  OfficeWorker     Fixed-18:00  reward     2.01  ctr 0.316  sends   7.00  optout 0.0
00
  OfficeWorker     Random       reward  -121.53  ctr 0.070  sends  29.14  optout 0.9
95
  OfficeWorker     LinUCB       reward   -22.85  ctr 0.126  sends  39.19  optout 0.0
38
  NightOwlStudent  Fixed-18:00  reward   -18.14  ctr 0.060  sends   7.00  optout 0.0
00
  NightOwlStudent  Random       reward  -132.32  ctr 0.058  sends  31.00  optout 0.9
95
  NightOwlStudent  LinUCB       reward    -9.32  ctr 0.213  sends  19.88  optout 0.0
34

OfficeWorker
      agent  reward_mean  reward_sem_std      ctr  sends_per_episode  optout_rate
Fixed-18:00        2.015           0.000    0.316              7.000        0.000
     LinUCB      -22.849          22.885    0.126             39.192        0.038
     Random     -121.531           0.000    0.070             29.145        0.995

NightOwlStudent
      agent  reward_mean  reward_sem_std      ctr  sends_per_episode  optout_rate
     LinUCB       -9.321          18.775    0.213             19.883        0.034
Fixed-18:00      -18.138           0.000    0.060              7.000        0.000
     Random     -132.320           0.000    0.058             31.000        0.995
```

In [20]:

```python
# --- 8.3 Watching a policy learn ---------------------------------------------
# One policy trained on the mixed population, probed separately against each
# focus archetype. The snapshots also drive the animation in the final section.

TRACE = train_and_evaluate(
    lambda s: LinUCBAgent(encoder_name="onehot", alpha=0.5, seed=s),
    n_seeds=1, archetype=None, snapshot_every=50)

snaps = TRACE["snapshots"]
eps = [e for e, _ in snaps]

fig, axes = plt.subplots(1, 3, figsize=(12.5, 3.1))
for arch in FOCUS:
    axes[0].plot(eps, [s[arch]["reward"] for _, s in snaps],
                 color=ARCH_COLOUR[arch], lw=1.8, label=arch)
    axes[1].plot(eps, [s[arch]["sends"] for _, s in snaps],
                 color=ARCH_COLOUR[arch], lw=1.8)
    axes[2].plot(eps, [s[arch]["ctr"] for _, s in snaps],
                 color=ARCH_COLOUR[arch], lw=1.8)
axes[0].axhline(0.0, color="#999999", lw=0.8, ls=":")
axes[0].set_ylabel("greedy reward / week")
axes[1].set_ylabel("sends / week")
axes[2].set_ylabel("CTR")
for ax in axes:
    ax.set_xlabel("training episodes")
axes[0].legend(frameon=False, fontsize=8)
fig.suptitle("G5   LinUCB trained on the mixed population, probed per archetype", f
fig.tight_layout()
save(fig, "G5_learning_curve")
plt.show()
```

```
for arch in FOCUS:
    a, b = snaps[0][1][arch], snaps[-1][1][arch]
    print(f"{arch:17} reward {a['reward']:+7.2f} -> {b['reward']:+7.2f}   "
          f"sends {a['sends']:5.1f} -> {b['sends']:5.1f}   "
          f"ctr {a['ctr']:.3f} -> {b['ctr']:.3f}")
```




```
OfficeWorker       reward -114.73 -> -100.45   sends  27.1 ->  64.8   ctr 0.070 -> 0.
144
NightOwlStudent    reward -140.26 ->  -86.25   sends  33.0 ->  32.2   ctr 0.053 -> 0.
054
```

---

9. Experiments

Three studies, all automated so that re-running regenerates every number and figure from scratch:

1. **Feature ablation** — Raw vs Harmonic vs One-hot, against both baselines. Establishes whether LinUCB's performance is limited by its *encoding* or by the bandit formulation itself.
2. **Exploration sweep** — `alpha` across its range, including `alpha = 0` (greedy ridge regression), isolating the contribution of exploration.
3. **Per-archetype breakdown** — where a stateless policy suffices and where it fails, which is the substance of RQ3.

Every configuration is trained on its own episodes and then scored on the same reserved `EVAL_SEEDS` under greedy action selection, so the reported numbers reflect learned policy rather than exploration noise.

In [21]:
```
# --- 9.1 Feature ablation ------------------------------------------------

ALPHA_DEFAULT = 0.5

# --- Machine-readable export ---------------------------------------------
# One row per (archetype, agent, seed) in the 7-column schema shared by
# all four notebooks, so the result files can be concatenated directly
# for the cross-algorithm comparison and the dashboard.
rows = []
for enc in ["raw", "harmonic", "onehot"]:
    rows.append(train_and_evaluate(
        lambda s, e=enc: LinUCBAgent(encoder_name=e, alpha=ALPHA_DEFAULT, seed=s),
        collect_hours=True))
```

```
rows.append(evaluate_baseline(lambda s: FixedScheduleAgent(hour=18)))
rows.append(evaluate_baseline(lambda s: RandomAgent(seed=s)))

ablation = pd.DataFrame([{k: v for k, v in r.items()
                          if k not in ("per_seed_rewards", "hour_actions")}
                         for r in rows])
ablation = ablation.sort_values("reward_mean", ascending=False).reset_index(drop=Tr

print(f"Feature ablation  (alpha={ALPHA_DEFAULT}, {TRAIN_EPISODES} train episodes,
      f"5 seeds, {len(EVAL_SEEDS)} held-out episodes)\n")
print(ablation.to_string(index=False, float_format=lambda v: f"{v:8.3f}"))
```

```
Feature ablation  (alpha=0.5, 600 train episodes, 5 seeds, 200 held-out episodes)

          agent  reward_mean  reward_std  reward_sem_std      ctr  sends_per_episode
optout_rate
     LinUCB-raw        0.000       0.000           0.000    0.000              0.000
0.000
    Fixed-18:00       -8.188      12.983           0.000    0.186              7.000
0.000
  LinUCB-onehot      -35.860      44.065          49.266    0.059             20.354
0.059
LinUCB-harmonic      -70.811      58.007          64.854    0.067             33.860
0.301
         Random     -117.479      74.885           0.000    0.074             28.415
1.000
```

In [22]:

```
# --- 9.2 Why LinUCB over-sends: a break-even analysis -----------------------
# The ablation shows LinUCB sending far more than either baseline and scoring
# poorly. This section establishes WHY, and separates three candidate
# explanations: a bug, a miscalibrated environment, or the bandit formulation.

cfg = CANE_CONFIG

# A single send raises fatigue by kappa, which then decays geometrically. The
# total future fatigue penalty it incurs is the sum of that decaying tail:
#     sum_t W_fat * kappa * lam^t  =  W_fat * kappa / (1 - lam)
rows = []
for arch in ARCHETYPES:
    for a in (ENGAGE, INCENTIVE):
        debt = cfg["W_fat"] * cfg["kappa"][a] / (1.0 - cfg["lam"])
        w = cfg["w_arch"][arch][a]
        rows.append({"archetype": arch, "action": ACTION_NAMES[a],
                     "fatigue_debt": debt, "send_cost": cfg["W_send"][a],
                     "true_breakeven": (debt + cfg["W_send"][a]) / (cfg["R_click"]
                     "myopic_breakeven": cfg["W_send"][a] / (cfg["R_click"] * w),
                     "peak_p0": ARCHETYPE_TABLE[arch].max()})
breakeven = pd.DataFrame(rows)
breakeven["ratio"] = breakeven["true_breakeven"] / breakeven["myopic_breakeven"]
breakeven["reachable"] = breakeven["peak_p0"] > breakeven["true_breakeven"]

print("Break-even click probability, per archetype and message type")
print()
print(breakeven.to_string(index=False, float_format=lambda v: f"{v:8.3f}"))
print()
```

```
print("LinUCB optimises the myopic column; a full-MDP agent optimises the true one,
print(f"so LinUCB treats a send as worthwhile across a {breakeven['ratio'].mean():.
print("range of contexts than it should.")
print()
print("'reachable' = the archetype's best hour clears its own break-even, i.e. that
print("message type is ever worth sending to that user at all.")
print()

# --- Oracle calibration ------------------------------------------------------
# An oracle with privileged access to the true click probability, sending only
# when p exceeds a threshold. Sweeping the threshold shows (a) that the
# environment is solvable and what the achievable ceiling is, and (b) what score
# a perfectly-informed agent gets if it uses LinUCB's myopic threshold.

class ThresholdOracle(Agent):
    """Sends when the TRUE click probability exceeds `thresh`.

    Not a competitor: it reads hidden environment state (archetype, activity)
    that no agent in the comparison can see. It exists to calibrate the task.
    """

    def __init__(self, env, thresh):
        self.env, self.thresh = env, thresh

    @property
    def name(self):
        return f"Oracle(p>{self.thresh:.2f})"

    def act(self, state, greedy=False):
        e = self.env
        if e.active:
            return HOLD, {}
        p = {a: min(e._p0() * (1.0 - cfg["mu"] * e.fatigue)
                    * cfg["w_arch"][e.archetype][a], 1.0)
             for a in (ENGAGE, INCENTIVE)}
        # Pick the better-value message type, but send only if its click
        # probability clears the swept threshold: the threshold is the control.
        best = max(p, key=lambda a: p[a] * cfg["R_click"] - cfg["W_send"][a]
                   - cfg["W_fat"] * cfg["kappa"][a] / (1.0 - cfg["lam"]))
        return (best if p[best] > self.thresh else HOLD), {}

    def update(self, *a, **k):
        return {}


oracle_rows = []
for th in [0.05, 0.10, 0.20, 0.30, 0.40, 0.50, 0.60]:
    env_o = CANEEnv(seed=7000)
    m, _, _, _ = run_episodes(ThresholdOracle(env_o, th), env_o,
                              seeds=EVAL_SEEDS, learn=False, greedy=True)
    oracle_rows.append(m)

oracle = pd.DataFrame(oracle_rows)[
    ["agent", "reward_mean", "ctr", "sends_per_episode", "optout_rate"]]
print("Oracle threshold sweep (privileged access; calibration only)\n")
print(oracle.to_string(index=False, float_format=lambda v: f"{v:8.3f}"))
```

```
best = oracle.loc[oracle["reward_mean"].idxmax()]
lo = oracle.iloc[0]
print()
print(f"Ceiling: {best['agent']} reaches {best['reward_mean']:+.1f} with "
      f"{best['sends_per_episode']:.1f} sends/week, so the task is solvable and wel
print(f"At the myopic end, {lo['agent']} sends {lo['sends_per_episode']:.1f} times/
      f"{lo['reward_mean']:+.1f}")
print(f"-- a {best['reward_mean'] - lo['reward_mean']:+.1f} reward difference from
      f"decision RULE alone, since both oracles")
print("see the identical privileged information. The deficit is therefore not attri
print("to the encoding, the implementation, or the environment.")
```

```
Break-even click probability, per archetype and message type

       archetype    action  fatigue_debt  send_cost  true_breakeven  myopic_breakeve
n  peak_p0    ratio  reachable
    OfficeWorker    Engage         2.400      1.000           0.296             0.08
7    0.485    3.400       True
    OfficeWorker Incentive         3.600      2.500           0.581             0.23
8    0.485    2.440      False
 NightOwlStudent    Engage         2.400      1.000           0.378             0.11
1    0.503    3.400       True
 NightOwlStudent Incentive         3.600      2.500           0.381             0.15
6    0.503    2.440       True
NightShiftWorker    Engage         2.400      1.000           0.340             0.10
0    0.429    3.400       True
NightShiftWorker Incentive         3.600      2.500           0.488             0.20
0    0.429    2.440      False
   NormalStudent    Engage         2.400      1.000           0.378             0.11
1    0.452    3.400       True
   NormalStudent Incentive         3.600      2.500           0.394             0.16
1    0.452    2.440       True
       Housewife    Engage         2.400      1.000           0.324             0.09
5    0.529    3.400       True
       Housewife Incentive         3.600      2.500           0.421             0.17
2    0.529    2.440       True

LinUCB optimises the myopic column; a full-MDP agent optimises the true one,
so LinUCB treats a send as worthwhile across a 2.9x wider
range of contexts than it should.

'reachable' = the archetype's best hour clears its own break-even, i.e. that
message type is ever worth sending to that user at all.

Oracle threshold sweep (privileged access; calibration only)

         agent  reward_mean      ctr  sends_per_episode  optout_rate
Oracle(p>0.05)     -107.812    0.130             56.715        0.625
Oracle(p>0.10)      -37.356    0.211             57.425        0.105
Oracle(p>0.20)       18.660    0.315             35.335        0.005
Oracle(p>0.30)       30.897    0.417             20.730        0.000
Oracle(p>0.40)       20.873    0.516             10.665        0.000
Oracle(p>0.50)       15.291    0.639              5.855        0.000
Oracle(p>0.60)        7.355    0.712              3.195        0.000

Ceiling: Oracle(p>0.30) reaches +30.9 with 20.7 sends/week, so the task is solvable
and well posed.
At the myopic end, Oracle(p>0.05) sends 56.7 times/week for -107.8
-- a +138.7 reward difference from the decision RULE alone, since both oracles
see the identical privileged information. The deficit is therefore not attributable
to the encoding, the implementation, or the environment.
```

In [23]:
```
# --- 9.3 Does one policy adapt to the different user types? (RQ3) -----------
# The proposal asks whether a SINGLE learned policy paces itself differently for
# each archetype without per-user retraining. For LinUCB there is a structural
# reason to doubt it, and this section measures the size of the problem.
#
# The observation is s = [hour, day, active, fatigue, recency]. NOTHING in it
```

```
# records how this particular user responded to previous sends. An Office Worker
# and a Night-Shift Worker standing at 18:00 with the same fatigue present the
# agent with IDENTICAL vectors, yet want opposite actions. A memoryless policy
# therefore cannot identify who it is talking to; it can only learn a single
# population-average schedule.
#
# The experiment isolates that cost:
#   shared      - one policy trained on the mixed population, scored per archetype
#   specialist  - a policy trained on ONE archetype only. It cannot be confused
#                 about who it is serving, so it is the upper bound achievable
#                 at this capacity.
#   gap         - specialist - shared = the price of not knowing the user.

ARCH_ENCODER = "onehot"     # max capacity, so any gap reflects identification,
                            # not an inability to represent the hourly pattern
ARCH_SEEDS = 5


def train_agents(encoder_name, alpha, n_seeds, archetype=None,
                 train_episodes=TRAIN_EPISODES):
    """Train `n_seeds` agents and return them (without evaluating)."""
    agents = []
    for seed in range(n_seeds):
        ag = LinUCBAgent(encoder_name=encoder_name, alpha=alpha, seed=seed)
        run_episodes(ag, CANEEnv(seed=1000 + seed, archetype=archetype),
                     n_episodes=train_episodes, learn=True, greedy=False)
        agents.append(ag)
    return agents


def eval_on(agents, archetype):
    """Greedy evaluation of pre-trained agents on one archetype."""
    per_seed = []
    for i, ag in enumerate(agents):
        env = CANEEnv(seed=7000 + i, archetype=archetype)
        m, _, _, _ = run_episodes(ag, env, seeds=EVAL_SEEDS, learn=False, greedy=Tr
        per_seed.append(m)
    return (np.mean([m["reward_mean"] for m in per_seed]),
            np.mean([m["ctr"] for m in per_seed]),
            np.mean([m["sends_per_episode"] for m in per_seed]),
            np.mean([m["optout_rate"] for m in per_seed]))


# One shared policy, trained on the mixed population (the RQ3 condition).
shared_agents = train_agents(ARCH_ENCODER, ALPHA_DEFAULT, ARCH_SEEDS)

arch_rows = []
for arch in ARCHETYPES:
    r_shared, ctr_s, sends_s, out_s = eval_on(shared_agents, arch)

    # Specialist: same algorithm, same capacity, never sees another archetype.
    specialists = train_agents(ARCH_ENCODER, ALPHA_DEFAULT, ARCH_SEEDS, archetype=a
    r_spec, ctr_p, sends_p, out_p = eval_on(specialists, arch)

    # Fixed baseline restricted to this archetype, for context.
    m_fix, _, _, _ = run_episodes(FixedScheduleAgent(18),
```

```
                                    CANEEnv(seed=7000, archetype=arch),
                                    seeds=EVAL_SEEDS, learn=False, greedy=True)

    arch_rows.append({
        "archetype": arch,
        "shared": r_shared, "specialist": r_spec, "gap": r_spec - r_shared,
        "fixed18": m_fix["reward_mean"],
        "ctr_shared": ctr_s, "ctr_spec": ctr_p,
        "sends_shared": sends_s, "sends_spec": sends_p,
        "optout_shared": out_s,
    })

archetype_df = pd.DataFrame(arch_rows)
print(f"Per-archetype performance  (LinUCB-{ARCH_ENCODER}, alpha={ALPHA_DEFAULT}, "
      f"{ARCH_SEEDS} seeds)\n")
print(archetype_df.to_string(index=False, float_format=lambda v: f"{v:8.2f}"))
print(f"\nMean cost of not knowing the user: "
      f"{archetype_df['gap'].mean():+.1f} reward/episode")
```

```
Per-archetype performance  (LinUCB-onehot, alpha=0.5, 5 seeds)

       archetype   shared  specialist      gap  fixed18  ctr_shared  ctr_spec  sends
_shared  sends_spec  optout_shared
    OfficeWorker   -38.94      -22.85    16.10     2.01        0.06      0.13
22.90       39.19           0.12
 NightOwlStudent   -34.13       -9.32    24.81   -18.14        0.02      0.21
11.70       19.88           0.00
NightShiftWorker   -43.39      -12.87    30.52    -4.18        0.06      0.04
21.11       12.93           0.04
   NormalStudent   -42.99      -25.47    17.52    -7.39        0.05      0.10
20.71       17.11           0.03
       Housewife   -19.10      -17.12     1.98    -7.62        0.09      0.12
24.76       21.05           0.18

Mean cost of not knowing the user: +18.2 reward/episode
```

10. Does the belief state pay for itself?

The state change is only justified if it buys performance. Three encodings of the same agent on the same task: the original memoryless five features, the belief-augmented state, and the belief state with hour-by-activity interaction terms. The archetype-specialist rows are the ceiling -- what the agent could do if it were simply told which user it faces.

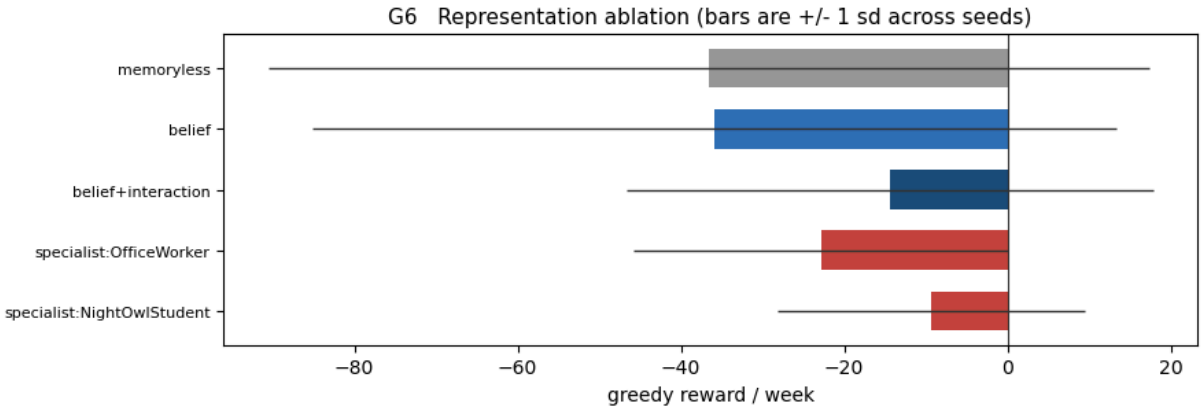
In [24]:
```
# --- 10.1 Representation ablation ------------------------------------------

from contextlib import contextmanager

@contextmanager
def belief_state(flag):
    """Temporarily switch the belief features on or off.

    Encoders read USE_BELIEF when called, and LinUCBAgent sizes itself from the
    encoder at construction, so agents must be built inside the block.
    """
```

```
    global USE_BELIEF
    prev = USE_BELIEF
    USE_BELIEF = flag
    try:
        yield
    finally:
        USE_BELIEF = prev


def _linucb(enc, label):
    return lambda s: LinUCBAgent(encoder_name=enc, alpha=0.5, seed=s, label=label)


REPR = {}
with belief_state(False):
    REPR["memoryless"] = train_and_evaluate(_linucb("onehot", "memoryless"),
                                            collect_hours=True)
with belief_state(True):
    REPR["belief"] = train_and_evaluate(_linucb("onehot", "belief"),
                                        collect_hours=True)
    REPR["belief+interaction"] = train_and_evaluate(
        _linucb("interaction", "belief+interaction"), collect_hours=True)
    for arch in FOCUS:
        REPR[f"specialist:{arch}"] = train_and_evaluate(
            _linucb("onehot", f"specialist:{arch}"), archetype=arch)

repr_table = pd.DataFrame([
    {"representation": k, "reward_mean": v["reward_mean"],
     "reward_sem_std": v["reward_sem_std"], "ctr": v["ctr"],
     "sends_per_episode": v["sends_per_episode"], "optout_rate": v["optout_rate"]}
    for k, v in REPR.items()])

print("Representation ablation  (LinUCB, mixed population unless specialist)")
print()
print(repr_table.to_string(index=False, float_format=lambda v: f"{v:8.3f}"))
```

```
Representation ablation  (LinUCB, mixed population unless specialist)

            representation  reward_mean  reward_sem_std      ctr  sends_per_episode
optout_rate
                memoryless      -36.608          53.987    0.056             22.393
0.042
                    belief      -35.860          49.266    0.059             20.354
0.059
       belief+interaction      -14.441          32.291    0.033             11.426
0.017
   specialist:OfficeWorker      -22.849          22.885    0.126             39.192
0.038
specialist:NightOwlStudent       -9.321          18.775    0.213             19.883
0.034
```

In [25]:
```
# --- 10.2 G6  What the representation is worth ------------------------------

keys = list(REPR)
vals = [REPR[k]["reward_mean"] for k in keys]
errs = [REPR[k]["reward_sem_std"] for k in keys]
```

```
cols = ["#999999", "#2f6fb5", "#1a4b7a"] + ["#c4453c"] * (len(keys) - 3)

fig, ax = plt.subplots(figsize=(9, 3.2))
ax.barh(range(len(keys)), vals, xerr=errs, color=cols[:len(keys)],
        height=0.65, error_kw={"lw": 1.0, "ecolor": "#333333"})
ax.axvline(0.0, color="#333333", lw=0.8)
ax.set_yticks(range(len(keys)))
ax.set_yticklabels(keys, fontsize=8)
ax.invert_yaxis()
ax.set_xlabel("greedy reward / week")
ax.set_title("G6   Representation ablation (bars are +/- 1 sd across seeds)", fonts
fig.tight_layout()
save(fig, "G6_representation_ablation")
plt.show()

gain = REPR["belief"]["reward_mean"] - REPR["memoryless"]["reward_mean"]
inter = REPR["belief+interaction"]["reward_mean"] - REPR["belief"]["reward_mean"]
print(f"belief features:      {gain:+.2f} reward/week over memoryless")
print(f"interaction terms:    {inter:+.2f} further")
```




```
belief features:      +0.75 reward/week over memoryless
interaction terms:    +21.42 further
```

11. Comparison

Everything below iterates the registry, so each agent your teammates add with a single `register(...)` call appears in every table and figure without any change here.

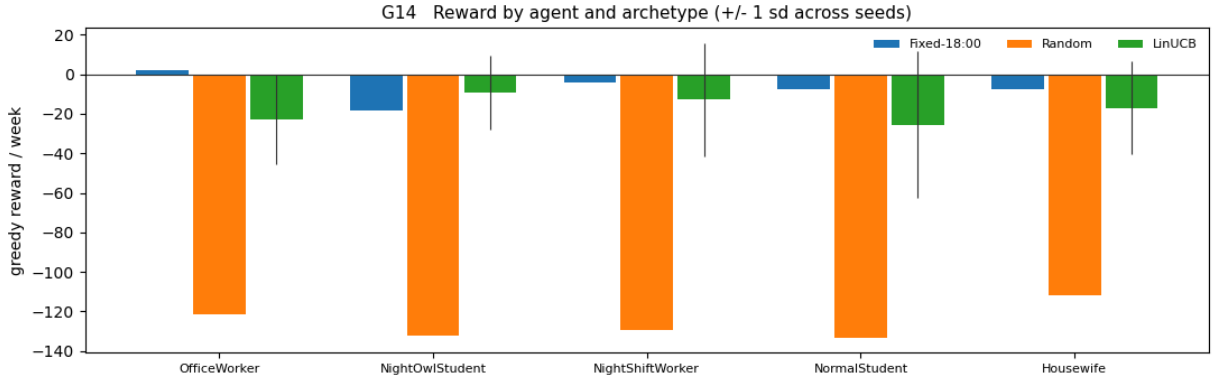
In [26]:
```
# --- 11.1 Every agent against every archetype ------------------------------

# --- Evaluation sweep ------------------------------------------------------
# Every agent against every archetype. Per-archetype rather than pooled:
# averaging over the five user types hides the result, because a policy
# can be correct for one archetype and clearly wrong for another.
RESULTS_ALL = {}
for arch in ARCHETYPES:
    RESULTS_ALL[arch] = run_all(arch)

summary = pd.DataFrame([
    {"archetype": arch, "agent": name, "reward": r["reward_mean"],
     "sd": r["reward_sem_std"], "ctr": r["ctr"],
     "sends": r["sends_per_episode"], "optout": r["optout_rate"]}
```

```
    for arch, res in RESULTS_ALL.items() for name, r in res.items()])

print()
print(summary.pivot(index="agent", columns="archetype", values="reward")
      .to_string(float_format=lambda v: f"{v:8.2f}"))
```

```
  OfficeWorker     Fixed-18:00  reward     2.01  ctr 0.316  sends   7.00  optout 0.0
00
  OfficeWorker     Random       reward  -121.53  ctr 0.070  sends  29.14  optout 0.9
95
  OfficeWorker     LinUCB       reward   -22.85  ctr 0.126  sends  39.19  optout 0.0
38
  NightOwlStudent  Fixed-18:00  reward   -18.14  ctr 0.060  sends   7.00  optout 0.0
00
  NightOwlStudent  Random       reward  -132.32  ctr 0.058  sends  31.00  optout 0.9
95
  NightOwlStudent  LinUCB       reward    -9.32  ctr 0.213  sends  19.88  optout 0.0
34
  NightShiftWorker Fixed-18:00  reward    -4.18  ctr 0.238  sends   7.00  optout 0.0
00
  NightShiftWorker Random       reward  -129.61  ctr 0.068  sends  31.14  optout 0.9
95
  NightShiftWorker LinUCB       reward   -12.87  ctr 0.036  sends  12.93  optout 0.0
08
  NormalStudent    Fixed-18:00  reward    -7.39  ctr 0.197  sends   7.00  optout 0.0
00
  NormalStudent    Random       reward  -133.41  ctr 0.064  sends  31.89  optout 0.9
95
  NormalStudent    LinUCB       reward   -25.47  ctr 0.100  sends  17.11  optout 0.0
47
  Housewife        Fixed-18:00  reward    -7.62  ctr 0.194  sends   7.00  optout 0.0
00
  Housewife        Random       reward  -111.60  ctr 0.098  sends  29.13  optout 0.9
90
  Housewife        LinUCB       reward   -17.12  ctr 0.118  sends  21.05  optout 0.1
14

archetype    Housewife  NightOwlStudent  NightShiftWorker  NormalStudent  OfficeWork
er
agent
Fixed-18:00      -7.62           -18.14             -4.18          -7.39          2.
01
LinUCB           -17.12            -9.32            -12.87         -25.47        -22.
85
Random          -111.60          -132.32           -129.61        -133.41       -121.
53
```

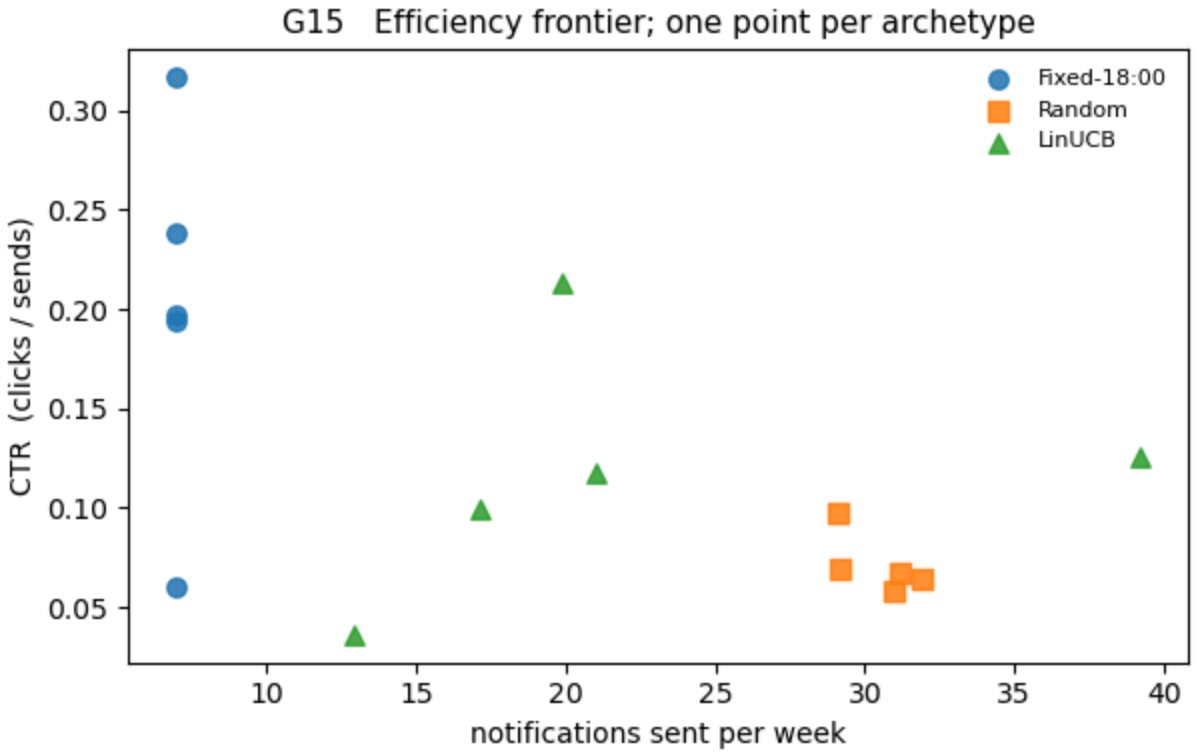
In [27]:
```
# --- 11.2 G14  Reward by agent and archetype --------------------------------

names = list(AGENTS)
x = np.arange(len(ARCHETYPES))
width = 0.8 / len(names)

fig, ax = plt.subplots(figsize=(11, 3.6))
for i, name in enumerate(names):
    vals = [RESULTS_ALL[a][name]["reward_mean"] for a in ARCHETYPES]
```

```
    errs = [RESULTS_ALL[a][name]["reward_sem_std"] for a in ARCHETYPES]
    ax.bar(x + i * width - 0.4 + width / 2, vals, width * 0.92, yerr=errs,
           label=name, error_kw={"lw": 0.8, "ecolor": "#333333"})
ax.axhline(0.0, color="#333333", lw=0.8)
ax.set_xticks(x)
ax.set_xticklabels(ARCHETYPES, fontsize=8)
ax.set_ylabel("greedy reward / week")
ax.legend(frameon=False, fontsize=8, ncol=len(names))
ax.set_title("G14   Reward by agent and archetype (+/- 1 sd across seeds)", fontsiz
fig.tight_layout()
save(fig, "G14_reward_by_archetype")
plt.show()
```



In [28]:

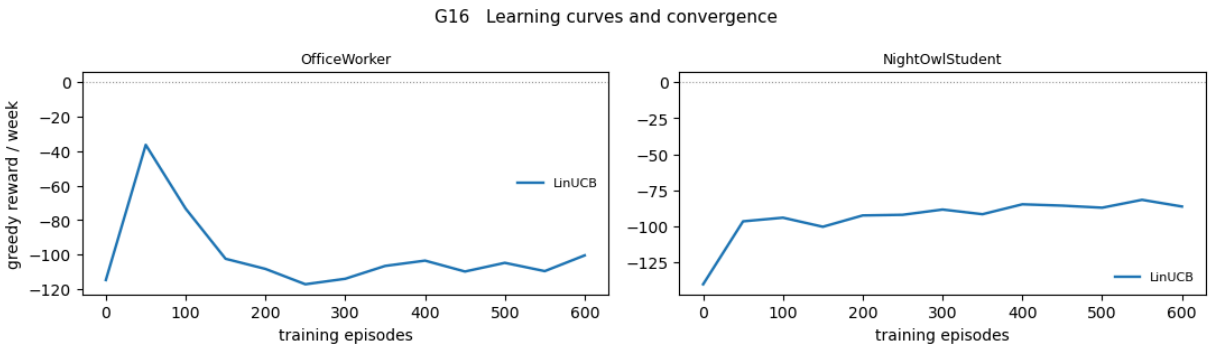
```
# --- 11.3 G15  Efficiency frontier ----------------------------------------
# A good agent is down-and-right: few sends, high click rate. Over-senders sit
# far right with a low CTR.

fig, ax = plt.subplots(figsize=(6.2, 4.0))
markers = ["o", "s", "^", "D", "v", "P", "X"]
for i, name in enumerate(AGENTS):
    sends = [RESULTS_ALL[a][name]["sends_per_episode"] for a in ARCHETYPES]
    ctr = [RESULTS_ALL[a][name]["ctr"] for a in ARCHETYPES]
    ax.scatter(sends, ctr, s=46, marker=markers[i % len(markers)], label=name, alph
ax.set_xlabel("notifications sent per week")
ax.set_ylabel("CTR  (clicks / sends)")
ax.legend(frameon=False, fontsize=8)
ax.set_title("G15   Efficiency frontier; one point per archetype", fontsize=11)
fig.tight_layout()
save(fig, "G15_efficiency_frontier")
plt.show()
```



In [29]:

```
# --- 11.4 G16  Convergence speed ---------------------------------------------
# Episodes needed to first reach 90% of the policy's own final reward. Only the
# learning agents are instrumented, and only one seed each: a snapshot costs
# probe episodes, so this measures shape, not a seed-averaged quantity.

CONVERGENCE = {}
for name, spec in AGENTS.items():
    if spec["kind"] == "baseline":
        continue
    r = train_and_evaluate(spec["factory"], n_seeds=1, snapshot_every=50)
    CONVERGENCE[name] = r["snapshots"]

fig, axes = plt.subplots(1, len(FOCUS), figsize=(11, 3.2), sharex=True)
# --- Machine-readable export -------------------------------------------------
# One row per (archetype, agent, seed) in the 7-column schema shared by
# all four notebooks, so the result files can be concatenated directly
# for the cross-algorithm comparison and the dashboard.
rows = []
for ax, arch in zip(np.atleast_1d(axes), FOCUS):
    for name, snaps in CONVERGENCE.items():
        eps = [e for e, _ in snaps]
        rew = np.array([s[arch]["reward"] for _, s in snaps])
        ax.plot(eps, rew, lw=1.7, label=name)
        final = rew[-1]
        target = final - 0.1 * abs(final)
        hit = np.where(rew >= target)[0] if final > rew[0] else np.array([len(rew)
        rows.append({"agent": name, "archetype": arch,
                     "final_reward": final,
                     "episodes_to_90pct": eps[int(hit[0])] if len(hit) else np.nan}
    ax.axhline(0.0, color="#999999", lw=0.8, ls=":")
    ax.set_title(arch, fontsize=9)
    ax.set_xlabel("training episodes")
```

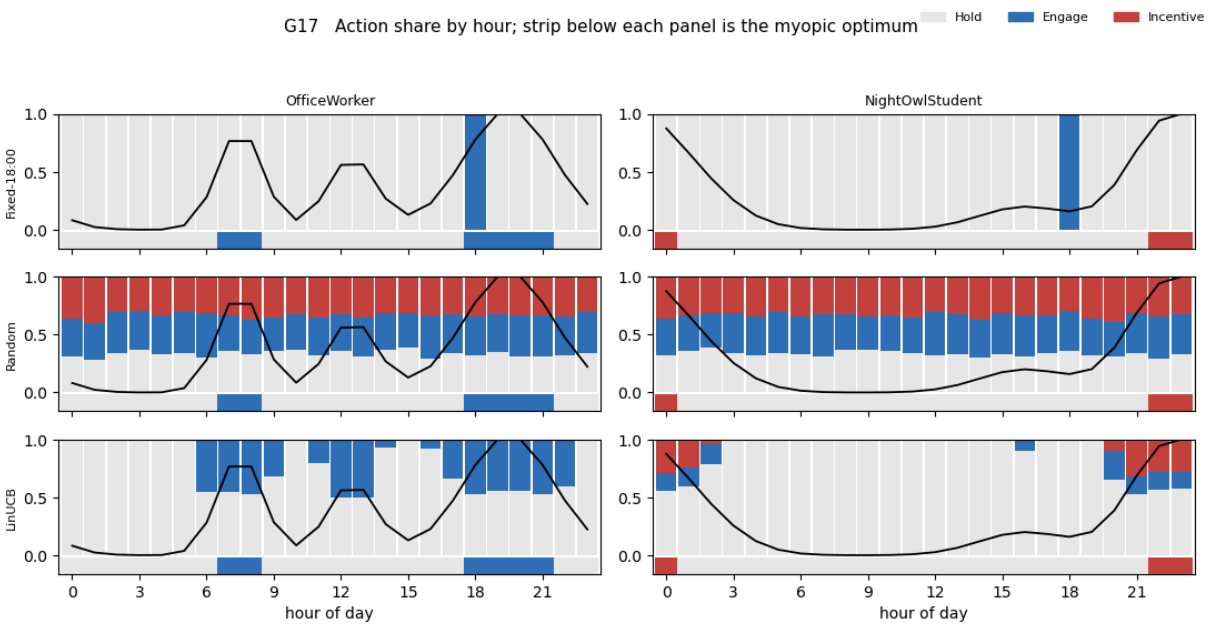
```
    ax.legend(frameon=False, fontsize=8)
np.atleast_1d(axes)[0].set_ylabel("greedy reward / week")
fig.suptitle("G16   Learning curves and convergence", fontsize=11)
fig.tight_layout()
save(fig, "G16_convergence")
plt.show()

print()
print(pd.DataFrame(rows).to_string(index=False, float_format=lambda v: f"{v:8.2f}"))
```



agent	archetype	final_reward	episodes_to_90pct
LinUCB	OfficeWorker	-100.45	50
LinUCB	NightOwlStudent	-86.25	100

In [30]:
```
# --- 11.5 G17  What each policy actually decides ----------------------------

from matplotlib.patches import Patch
# The behavioural evidence for RQ3. Bars are the action share at each hour; the
# black line is true receptiveness; the strip under each panel is the
# myopic-optimal action from 7.4, so a policy that has learned the context looks
# like the strip beneath it.

names = list(AGENTS)
fig, axes = plt.subplots(len(names), len(FOCUS),
                         figsize=(11, 1.9 * len(names)), squeeze=False,
                         sharex=True)
for r, name in enumerate(names):
    for c, arch in enumerate(FOCUS):
        ax = axes[r][c]
        ha = RESULTS_ALL[arch][name]["hour_actions"].astype(float)
        frac = ha / np.maximum(ha.sum(axis=1, keepdims=True), 1.0)
        bottom = np.zeros(24)
        for a, col in enumerate(ACTION_COLOUR):
            ax.bar(range(24), frac[:, a], bottom=bottom, color=col, width=0.92)
            bottom += frac[:, a]
        p0 = ARCHETYPE_TABLE[arch]
        ax.plot(range(24), p0 / p0.max(), color="black", lw=1.3)
        ax.imshow(GROUND_TRUTH[arch][0][None, :], aspect="auto",
                  cmap=ListedColormap(ACTION_COLOUR), vmin=0, vmax=2,
                  extent=[-0.5, 23.5, -0.16, -0.02])
        ax.set_ylim(-0.16, 1.0)
        ax.set_xlim(-0.6, 23.6)
        ax.set_xticks(range(0, 24, 3))
        if r == 0:
            ax.set_title(arch, fontsize=9)
        if c == 0:
```

```
            ax.set_ylabel(name, fontsize=8)
        if r == len(names) - 1:
            ax.set_xlabel("hour of day")

fig.legend(handles=[Patch(color=col, label=ACTION_NAMES[i])
                    for i, col in enumerate(ACTION_COLOUR)],
           loc="upper right", ncol=3, frameon=False, fontsize=8)
fig.suptitle("G17   Action share by hour; strip below each panel is the myopic opti
             fontsize=11)
fig.tight_layout(rect=[0, 0, 1, 0.94])
save(fig, "G17_decision_distribution")
plt.show()
```



12. Part B: optimisation, ensembling and robustness

The search harness, the ensemble wrapper and the perturbation suite are all agent-agnostic: they take anything implementing the `Agent` contract, so each teammate tunes their own algorithm through the same code path.

In [31]:
```
# --- 12.1 Random hyperparameter search ------------------------------------
# Random search rather than grid: for a fixed trial budget it covers each
# individual dimension far better, which matters when only some dimensions
# actually influence the result (Bergstra & Bengio, 2012).

def random_search(build, space, n_trials=12, seed=0, n_seeds=2,
                  train_episodes=None, archetype=None):
    """Sample n_trials configurations and score each.

    Args:
        build: (params, seed) -> Agent
        space: {name: list of choices, or callable(rng) -> value}
    """
    rng = np.random.default_rng(seed)
    # --- Machine-readable export ----------------------------------------------
```

```
    # One row per (archetype, agent, seed) in the 7-column schema shared by
    # all four notebooks, so the result files can be concatenated directly
    # for the cross-algorithm comparison and the dashboard.
    rows = []
    for t in range(n_trials):
        params = {k: (v(rng) if callable(v) else v[int(rng.integers(len(v)))])
                  for k, v in space.items()}
        r = train_and_evaluate(lambda s, p=params: build(p, s), n_seeds=n_seeds,
                               train_episodes=train_episodes or TRAIN_EPISODES,
                               archetype=archetype)
        rows.append({**params, "reward": r["reward_mean"], "ctr": r["ctr"],
                     "sends": r["sends_per_episode"], "optout": r["optout_rate"]})
        print(f"  trial {t + 1:2}/{n_trials}  {params}  ->  {r['reward_mean']:+.2f}
    return pd.DataFrame(rows).sort_values("reward", ascending=False).reset_index(dr


LINUCB_SPACE = {
    "alpha": lambda rng: round(float(rng.uniform(0.0, 1.5)), 3),
    "lam": lambda rng: round(float(10 ** rng.uniform(-1, 1)), 3),
    "encoder_name": ["onehot", "harmonic", "interaction"],
}

search = random_search(
    lambda p, s: LinUCBAgent(encoder_name=p["encoder_name"], alpha=p["alpha"],
                             lam=p["lam"], seed=s),
    LINUCB_SPACE, n_trials=6 if QUICK else 14)

print()
print(search.head(8).to_string(index=False, float_format=lambda v: f"{v:8.3f}"))
print()
print("Check the sends column: a configuration that never sends scores exactly 0,")
print("which beats any policy that sends badly. That is a real result, not a bug,")
print("but it is a degenerate optimum and should be reported as one.")
```

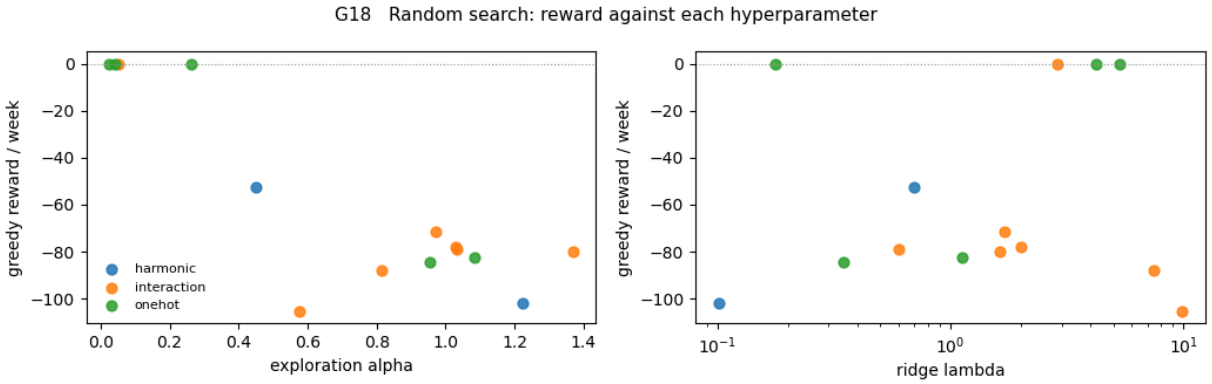
```
  trial  1/14  {'alpha': 0.955, 'lam': 0.346, 'encoder_name': 'onehot'}  ->  -84.67
  trial  2/14  {'alpha': 0.025, 'lam': 4.232, 'encoder_name': 'onehot'}  ->  +0.00
  trial  3/14  {'alpha': 1.369, 'lam': 1.634, 'encoder_name': 'interaction'}  ->  -8
0.21
  trial  4/14  {'alpha': 0.815, 'lam': 7.416, 'encoder_name': 'interaction'}  ->  -8
7.83
  trial  5/14  {'alpha': 1.224, 'lam': 0.101, 'encoder_name': 'harmonic'}  ->  -102.
20
  trial  6/14  {'alpha': 0.05, 'lam': 2.879, 'encoder_name': 'interaction'}  ->  +0.
00
  trial  7/14  {'alpha': 0.263, 'lam': 5.325, 'encoder_name': 'onehot'}  ->  +0.00
  trial  8/14  {'alpha': 0.45, 'lam': 0.7, 'encoder_name': 'harmonic'}  ->  -52.48
  trial  9/14  {'alpha': 0.042, 'lam': 0.177, 'encoder_name': 'onehot'}  ->  +0.00
  trial 10/14  {'alpha': 0.971, 'lam': 1.701, 'encoder_name': 'interaction'}  ->  -7
1.60
  trial 11/14  {'alpha': 0.576, 'lam': 9.872, 'encoder_name': 'interaction'}  ->  -1
05.33
  trial 12/14  {'alpha': 1.028, 'lam': 1.999, 'encoder_name': 'interaction'}  ->  -7
8.21
  trial 13/14  {'alpha': 1.033, 'lam': 0.6, 'encoder_name': 'interaction'}  ->  -79.
24
  trial 14/14  {'alpha': 1.082, 'lam': 1.124, 'encoder_name': 'onehot'}  ->  -82.46
```

alpha	lam	encoder_name	reward	ctr	sends	optout
0.025	4.232	onehot	0.000	0.000	0.000	0.000
0.050	2.879	interaction	0.000	0.000	0.000	0.000
0.263	5.325	onehot	0.000	0.000	0.000	0.000
0.042	0.177	onehot	0.000	0.000	0.000	0.000
0.450	0.700	harmonic	-52.476	0.061	27.660	0.140
0.971	1.701	interaction	-71.601	0.159	52.487	0.085
1.028	1.999	interaction	-78.208	0.157	56.587	0.098
1.033	0.600	interaction	-79.238	0.160	58.822	0.080

Check the sends column: a configuration that never sends scores exactly 0, which beats any policy that sends badly. That is a real result, not a bug, but it is a degenerate optimum and should be reported as one.

In [32]:
```
# --- 12.2 G18  What the search found ----------------------------------------

fig, axes = plt.subplots(1, 2, figsize=(10.5, 3.4))
for enc in sorted(search["encoder_name"].unique()):
    sel = search[search["encoder_name"] == enc]
    axes[0].scatter(sel["alpha"], sel["reward"], s=40, label=enc, alpha=0.85)
    axes[1].scatter(sel["lam"], sel["reward"], s=40, label=enc, alpha=0.85)
axes[0].set_xlabel("exploration alpha")
axes[1].set_xlabel("ridge lambda")
axes[1].set_xscale("log")
for ax in axes:
    ax.set_ylabel("greedy reward / week")
    ax.axhline(0.0, color="#999999", lw=0.8, ls=":")
axes[0].legend(frameon=False, fontsize=8)
fig.suptitle("G18   Random search: reward against each hyperparameter", fontsize=11
fig.tight_layout()
save(fig, "G18_random_search")
plt.show()
```

```
best = search.iloc[0]
print(f"best: encoder={best['encoder_name']}  alpha={best['alpha']}  "
      f"lam={best['lam']}  ->  {best['reward']:+.2f}")
```




```
best: encoder=onehot  alpha=0.025  lam=4.232  ->  +0.00
```

In [33]:

```
# --- 12.3 Ensemble ------------------------------------------------------------
# Majority vote over independently trained policies. With three actions and no
# majority the tie resolves to Hold: in a channel where a bad send costs more
# than a missed one, disagreement should default to silence.

class VotingEnsemble(Agent):
    """Wraps pre-trained agents; does not learn."""

    def __init__(self, agents, label="Ensemble(vote)"):
        self.agents = list(agents)
        self._label = label

    @property
    def name(self):
        return self._label

    def act(self, state, greedy=False):
        votes = np.bincount([a.act(state, greedy=True)[0] for a in self.agents],
                            minlength=N_ACTIONS)
        winners = np.flatnonzero(votes == votes.max())
        return (int(winners[0]) if len(winners) == 1 else HOLD), {}

    def update(self, *a, **k):
        return {}


def train_agent(factory, seed=0, train_episodes=None, archetype=None):
    """Train one agent and hand it back, ready to be ensembled or perturbed."""
    agent = factory(seed)
    env = CANEEnv(seed=1000 + seed, archetype=archetype)
    run_episodes(agent, env, n_episodes=train_episodes or TRAIN_EPISODES,
                 learn=True, greedy=False)
    return agent


# Members share a seed, so the only thing that differs between them is the
# algorithm or the representation -- not the luck of initialisation. Once more
# than one learning agent is registered the ensemble is built from those
```

```
# instead, which is where a vote actually has something to arbitrate.
_learners = [n for n, s in AGENTS.items() if s["kind"] == "learning"]
if len(_learners) >= 2:
    MEMBERS = [train_agent(AGENTS[n]["factory"], seed=0) for n in _learners]
else:
    MEMBERS = [train_agent(lambda s, e=enc: LinUCBAgent(encoder_name=e, alpha=0.5,
                           seed=0)
               for enc in ["onehot", "harmonic", "interaction"]]
print("members:", ", ".join(a.name for a in MEMBERS))

# --- Machine-readable export ------------------------------------------
# One row per (archetype, agent, seed) in the 7-column schema shared by
# all four notebooks, so the result files can be concatenated directly
# for the cross-algorithm comparison and the dashboard.
rows = []
for a in MEMBERS + [VotingEnsemble(MEMBERS)]:
    m, _, _, _ = run_episodes(a, CANEEnv(seed=7000), seeds=EVAL_SEEDS,
                              learn=False, greedy=True)
    rows.append(m)

ensemble_table = pd.DataFrame(rows)[
    ["agent", "reward_mean", "ctr", "sends_per_episode", "optout_rate"]]
print("Ensemble vs its constituents")
print()
print(ensemble_table.to_string(index=False, float_format=lambda v: f"{v:8.3f}"))
```

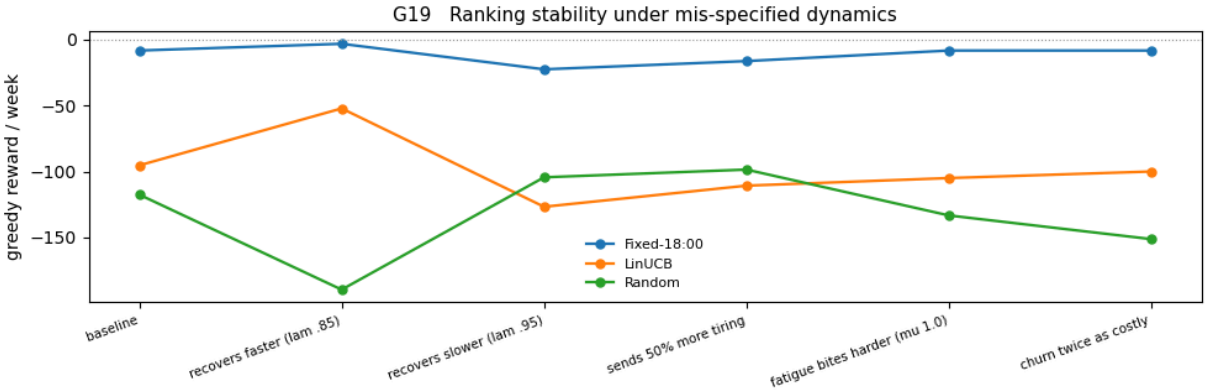
```
members: LinUCB-onehot, LinUCB-harmonic, LinUCB-interaction
Ensemble vs its constituents

             agent  reward_mean      ctr  sends_per_episode  optout_rate
     LinUCB-onehot      -95.311    0.130              54.495        0.155
   LinUCB-harmonic     -116.198    0.119              60.185        0.320
LinUCB-interaction      -72.205    0.165              57.130        0.085
    Ensemble(vote)      -97.265    0.134              55.885        0.290
```

In [34]:

```
# --- 12.4 Robustness to mis-specified dynamics ------------------------------
# The dynamics constants are modelling assumptions, not measurements. If the
# ranking only survives at the exact calibration it is an artefact, so every
# agent is re-scored under perturbed dynamics without being retrained.

PERTURBATIONS = {
    "baseline": {},
    "recovers faster (lam .85)": {"lam": 0.85},
    "recovers slower (lam .95)": {"lam": 0.95},
    "sends 50% more tiring": {"kappa": {HOLD: 0.0, ENGAGE: 0.18, INCENTIVE: 0.27}},
    "fatigue bites harder (mu 1.0)": {"mu": 1.0},
    "churn twice as costly": {"R_churn": 60.0},
}

TRAINED = {}
for name, spec in AGENTS.items():
    TRAINED[name] = (spec["factory"](0) if spec["kind"] == "baseline"
                     else train_agent(spec["factory"], seed=0))

rob_rows = []
for label, override in PERTURBATIONS.items():
```

```
    for name, agent in TRAINED.items():
        m, _, _, _ = run_episodes(agent, CANEEnv(config=override, seed=7000),
                                  seeds=EVAL_SEEDS, learn=False, greedy=True)
        rob_rows.append({"perturbation": label, "agent": name,
                         "reward": m["reward_mean"]})

robust = (pd.DataFrame(rob_rows).pivot(index="perturbation", columns="agent",
                                       values="reward").loc[list(PERTURBATIONS)])
print("Reward under perturbed dynamics (all agents trained on the baseline only)")
print()
print(robust.to_string(float_format=lambda v: f"{v:8.2f}"))

fig, ax = plt.subplots(figsize=(10, 3.4))
for name in robust.columns:
    ax.plot(range(len(robust)), robust[name], marker="o", lw=1.6, ms=5, label=name)
ax.axhline(0.0, color="#999999", lw=0.8, ls=":")
ax.set_xticks(range(len(robust)))
ax.set_xticklabels(robust.index, rotation=20, ha="right", fontsize=7.5)
ax.set_ylabel("greedy reward / week")
ax.legend(frameon=False, fontsize=8)
ax.set_title("G19   Ranking stability under mis-specified dynamics", fontsize=11)
fig.tight_layout()
save(fig, "G19_robustness")
plt.show()

spread = (robust.max() - robust.min()).sort_values()
print()
print("reward spread across perturbations (smaller = more robust):")
print(spread.to_string(float_format=lambda v: f"{v:8.2f}"))
```

```
Reward under perturbed dynamics (all agents trained on the baseline only)

agent                          Fixed-18:00   LinUCB   Random
perturbation
baseline                             -8.19   -95.31  -117.48
recovers faster (lam .85)            -3.08   -52.12  -189.58
recovers slower (lam .95)           -22.45  -126.71  -104.37
sends 50% more tiring               -16.20  -110.70   -98.51
fatigue bites harder (mu 1.0)        -8.19  -104.93  -133.30
churn twice as costly                -8.19   -99.96  -151.25
```



```
reward spread across perturbations (smaller = more robust):
agent
Fixed-18:00      19.37
LinUCB           74.60
Random           91.07
```

13. Statistical robustness

A ranking built from five noisy seeds is not a result until it survives a test. Each method's per-seed greedy rewards are summarised with a 95% interval and compared against each baseline with both a Welch t-test (unequal variances) and an exact Mann-Whitney U (no normality assumption).

In [ ]:
```python
# --- 13.1 Intervals and pairwise tests ------------------------------------
# Baselines are scored across the same number of seeds as the learners so the
# two samples are comparable; for a baseline the spread comes from the
# evaluation environment rather than from training.

from scipy import stats


def evaluate_across_seeds(make_agent, n_seeds=None, archetype=None):
    n_seeds = N_SEEDS if n_seeds is None else n_seeds
    out = []
    for seed in range(n_seeds):
        env = CANEEnv(seed=7000 + seed, archetype=archetype)
        m, _, _, _ = run_episodes(make_agent(seed), env, seeds=EVAL_SEEDS,
                                  learn=False, greedy=True)
        out.append(m["reward_mean"])
    return np.array(out)


PER_SEED = {}
for name, spec in AGENTS.items():
    PER_SEED[name] = (evaluate_across_seeds(spec["factory"])
                      if spec["kind"] == "baseline"
                      else train_and_evaluate(spec["factory"])["per_seed_rewards"])

# --- Machine-readable export ----------------------------------------------
# One row per (archetype, agent, seed) in the 7-column schema shared by
# all four notebooks, so the result files can be concatenated directly
# for the cross-algorithm comparison and the dashboard.
rows = []
for name, r in PER_SEED.items():
    n = len(r)
    sd = float(r.std(ddof=1)) if n > 1 else 0.0
    half = float(stats.t.ppf(0.975, n - 1)) * sd / np.sqrt(n) if n > 1 else np.nan
    rows.append({"agent": name, "n": n, "mean": r.mean(), "sd": sd,
                 "ci_low": r.mean() - half, "ci_high": r.mean() + half})

ci_table = pd.DataFrame(rows).sort_values("mean", ascending=False).reset_index(drop
print("95% confidence intervals (t-interval, sample sd, ddof=1)")
print()
```

```
print(ci_table.to_string(index=False, float_format=lambda v: f"{v:8.2f}"))
print()
print("A deterministic policy scored on the identical held-out episodes has no")
print("seed variance by construction, so its interval collapses to a point. That")
print("is a property of the baseline, not evidence that it is precisely estimated."
```

In [ ]:

```
# --- 13.2 Every learner against every baseline ----------------------------

def welch_p(a, b):
    """nan rather than a warning when both samples are constant."""
    if len(a) < 2 or len(b) < 2:
        return float("nan")
    if a.std(ddof=1) == 0 and b.std(ddof=1) == 0:
        return float("nan")
    return float(stats.ttest_ind(a, b, equal_var=False).pvalue)


learners = [n for n, s in AGENTS.items() if s["kind"] == "learning"]
baselines = [n for n, s in AGENTS.items() if s["kind"] == "baseline"]

# --- Machine-readable export ----------------------------------------------
# One row per (archetype, agent, seed) in the 7-column schema shared by
# all four notebooks, so the result files can be concatenated directly
# for the cross-algorithm comparison and the dashboard.
rows = []
for L in learners:
    for B in baselines:
        a, b = PER_SEED[L], PER_SEED[B]
        u = stats.mannwhitneyu(a, b, alternative="two-sided")
        rows.append({"learner": L, "baseline": B, "diff": a.mean() - b.mean(),
                     "welch_p": welch_p(a, b),
                     "mwu_U": float(u.statistic), "mwu_p": float(u.pvalue),
                     "ci_separated": bool(
                         ci_table.set_index("agent").loc[L, "ci_low"]
                         > ci_table.set_index("agent").loc[B, "ci_high"])})

tests = pd.DataFrame(rows)
print("Pairwise comparisons")
print()
print(tests.to_string(index=False, float_format=lambda v: f"{v:8.4f}"))

n = min(len(v) for v in PER_SEED.values())
floor = float(stats.mannwhitneyu(np.arange(n), np.arange(n) + n,
                                 alternative="two-sided").pvalue)
print()
print(f"With n = {n} per group the smallest attainable two-sided Mann-Whitney p")
print(f"is {floor:.4f}. U at its maximum therefore means complete rank separation")
print("-- every seed of one method beating every seed of the other -- not a")
print("stronger claim than the sample size can support. The Welch test is")
print("reported alongside it rather than in place of it.")
```

In [ ]:

```
# --- 13.3 G20  Intervals, ranked ------------------------------------------

order = ci_table.iloc[::-1].reset_index(drop=True)
fig, ax = plt.subplots(figsize=(8, 0.5 * len(order) + 1.6))
```

```
for i, row in order.iterrows():
    lo, hi = row["ci_low"], row["ci_high"]
    if np.isfinite(lo) and np.isfinite(hi):
        ax.plot([lo, hi], [i, i], color="#333333", lw=1.4)
        ax.plot([lo, lo, hi, hi], [i - 0.12, i + 0.12, i - 0.12, i + 0.12],
                ls="none", marker="|", color="#333333", ms=7)
    ax.plot(row["mean"], i, "o", ms=7,
            color="#2f6fb5" if AGENTS[row["agent"]]["kind"] == "learning" else "#99
ax.axvline(0.0, color="#c4453c", lw=1.0, ls=":")
ax.set_yticks(range(len(order)))
ax.set_yticklabels(order["agent"], fontsize=8)
ax.set_xlabel("greedy reward / week")
ax.set_title("G20   Mean with 95% confidence interval", fontsize=11)
fig.tight_layout()
save(fig, "G20_confidence_intervals")
plt.show()
```

14. Animation: watching the policy learn

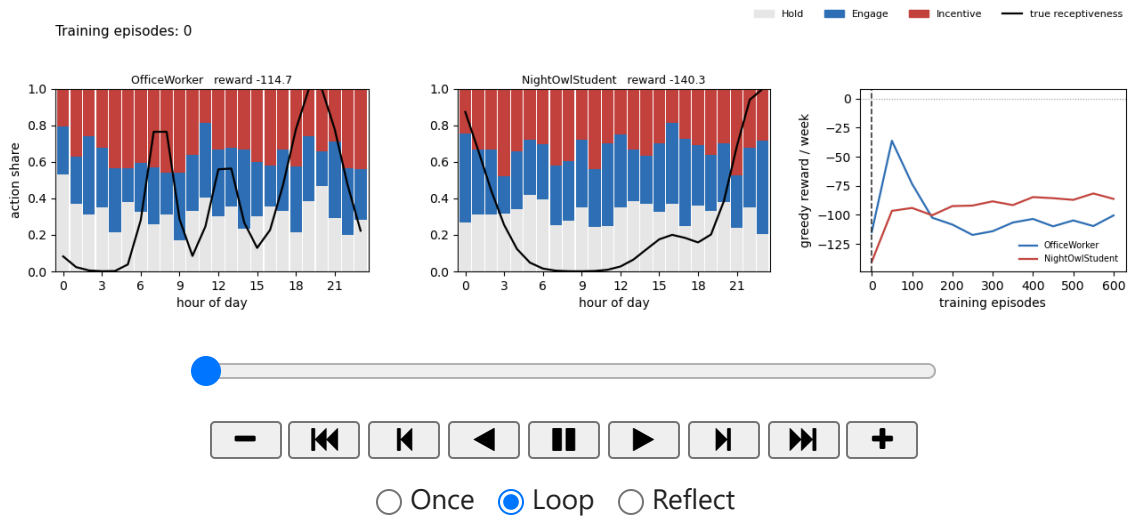
The snapshots from 8.3 replayed as frames. Each bar is one hour of the day, split by the action the frozen policy chooses there; the black line is the archetype's true receptiveness, which the agent never observes.

In [35]:
```
# --- 14.1 Policy evolution ---------------------------------------------------

import shutil
from IPython.display import HTML
from matplotlib.animation import FuncAnimation, PillowWriter
from matplotlib.patches import Patch

_ffmpeg = shutil.which("ffmpeg")
if _ffmpeg:
    plt.rcParams["animation.ffmpeg_path"] = _ffmpeg


def policy_animation(snaps, archetypes=FOCUS, tag="A1_policy_learning", fps=2):
    eps = [e for e, _ in snaps]
    lo = min(s[a]["reward"] for _, s in snaps for a in archetypes)
    hi = max(s[a]["reward"] for _, s in snaps for a in archetypes)
    pad = 0.08 * max(hi - lo, 1.0)

    fig, axes = plt.subplots(1, 3, figsize=(13, 3.8),
                             gridspec_kw={"width_ratios": [1, 1, 0.85]})
    fig.subplots_adjust(left=0.06, right=0.985, top=0.74, bottom=0.20, wspace=0.30)
    fig.legend(handles=[Patch(color=c, label=ACTION_NAMES[i])
                        for i, c in enumerate(ACTION_COLOUR)]
                       + [plt.Line2D([], [], color="black", lw=1.6,
                                     label="true receptiveness")],
               loc="upper right", ncol=4, frameon=False, fontsize=8,
               bbox_to_anchor=(0.99, 1.0))
    title = fig.text(0.06, 0.93, "", fontsize=11, va="top")

    def draw(k):
```

```
        ep, snap = snaps[k]
        for ax, arch in zip(axes[:2], archetypes):
            ax.clear()
            ha = snap[arch]["hour_actions"].astype(float)
            frac = ha / np.maximum(ha.sum(axis=1, keepdims=True), 1.0)
            bottom = np.zeros(24)
            for a, col in enumerate(ACTION_COLOUR):
                ax.bar(range(24), frac[:, a], bottom=bottom, color=col, width=0.92)
                bottom += frac[:, a]
            p0 = ARCHETYPE_TABLE[arch]
            ax.plot(range(24), p0 / p0.max(), color="black", lw=1.6)
            ax.set_xlim(-0.6, 23.6)
            ax.set_ylim(0, 1)
            ax.set_xticks(range(0, 24, 3))
            ax.set_xlabel("hour of day")
            ax.set_title(f"{arch}   reward {snap[arch]['reward']:+.1f}",
                         fontsize=9, pad=4)
        axes[0].set_ylabel("action share")

        ax = axes[2]
        ax.clear()
        for arch in archetypes:
            ax.plot(eps, [s[arch]["reward"] for _, s in snaps],
                    color=ARCH_COLOUR[arch], lw=1.6, label=arch)
        ax.axvline(ep, color="#333333", lw=1.2, ls="--")
        ax.axhline(0.0, color="#999999", lw=0.8, ls=":")
        ax.set_ylim(lo - pad, max(hi + pad, pad))
        ax.set_xlabel("training episodes")
        ax.set_ylabel("greedy reward / week")
        ax.legend(frameon=False, fontsize=7, loc="lower right")
        title.set_text(f"Training episodes: {ep}")

    anim = FuncAnimation(fig, draw, frames=len(snaps), interval=1000 // fps)
    anim.save(FIGDIR / f"{tag}.gif", writer=PillowWriter(fps=fps))
    try:
        anim.save(FIGDIR / f"{tag}.mp4", fps=fps)
        print(f"saved {tag}.gif and {tag}.mp4  ({len(snaps)} frames)")
    except Exception as exc:
        print(f"saved {tag}.gif  ({len(snaps)} frames; mp4 skipped: {type(exc).__na
    plt.close(fig)
    return anim


ANIM = policy_animation(snaps)
HTML(ANIM.to_jshtml())
```

```
saved A1_policy_learning.gif and A1_policy_learning.mp4  (13 frames)
```

Out[35]:



15. Machine-readable export

Every other arm of the comparison (DQN, Double DQN, PPO) writes a results CSV and saved model checkpoints. This notebook did not: its numbers existed only inside these output cells and in `run*.log`, which made the four-way comparison impossible to reproduce without re-running every notebook by hand.

The two cells below close that gap. `linucb_results.csv` uses the *same 7-column schema* the other notebooks emit, so the four files can be concatenated directly; because the evaluation protocol (`EVAL_SEEDS`, `TRAIN_EPISODES`, `N_SEEDS`) is shared across all four notebooks, the resulting comparison is like-for-like.

In [ ]:
```
# --- 15.1  Results export ------------------------------------------------
from pathlib import Path

_resdir = Path("results")
_resdir.mkdir(exist_ok=True)

_rows = []
for _arch, _res in RESULTS_ALL.items():
    for _name, _r in _res.items():
        _per_seed = np.atleast_1d(_r.get("per_seed_rewards", [_r["reward_mean"]]))
        for _si, _sr in enumerate(_per_seed):
            _rows.append({"archetype": _arch, "agent": _name, "seed": _si,
                          "reward_mean": float(_sr), "ctr": _r["ctr"],
                          "sends_per_episode": _r["sends_per_episode"],
                          "optout_rate": _r["optout_rate"]})

_df = pd.DataFrame(_rows)
_df.to_csv(_resdir / "linucb_results.csv", index=False)
print(f"Wrote {_resdir / 'linucb_results.csv'}  ({len(_df)} rows)")
print(_df.head(8).to_string(index=False))
```

In [ ]:
```
# --- 15.2  Model checkpointing -------------------------------------------
# LinUCB has no torch module: its entire learned state is the per-arm ridge
# statistics (A, A^-1, b), so it checkpoints to .npz rather than .pt. The
```

```
# dashboard's policy explorer reloads these to recompute UCB scores without
# retraining. Trained on the mixed population (no archetype argument), matching
# the checkpointing convention used by the DQN/DDQN/PPO notebooks.
_modeldir = Path("models")
_modeldir.mkdir(exist_ok=True)

for _seed in range(N_SEEDS):
    _agent = LinUCBAgent(encoder_name="onehot", alpha=0.5, seed=_seed)
    _train_env = CANEEnv(seed=1000 + _seed)
    run_episodes(_agent, _train_env, n_episodes=TRAIN_EPISODES,
                 learn=True, greedy=False)
    np.savez_compressed(
        _modeldir / f"linucb_seed{_seed}.npz",
        A=_agent.A, A_inv=_agent.A_inv, b=_agent.b, n_pulls=_agent.n_pulls,
        encoder_name=_agent.encoder_name, alpha=_agent.alpha,
        lam=_agent.lam, d=_agent.d, n_actions=_agent.n_actions,
    )
    print(f"Saved model: linucb_seed{_seed}.npz")

print("All LinUCB checkpoints saved.")
```

---

16. Key findings and implications

Results

Averaged over five training seeds and the 200 held-out evaluation episodes:

Archetype	Reward	CTR	Sends/week	Opt-out
Housewife	-17.12	0.118	21.0	0.114
NightOwlStudent	-9.32	0.213	19.9	0.034
NightShiftWorker	-12.87	0.036	12.9	0.008
NormalStudent	-25.47	0.100	17.1	0.047
OfficeWorker	-22.85	0.126	39.2	0.038
Mean	**-17.53**	**0.119**	**22.0**	**0.048**

Reference points: Fixed-18:00 averages **-7.06**, Random **-125.69**.

Observations and patterns

1. **LinUCB contacts all five users but loses money on every one of them.** Its mean CTR of 0.119 is roughly a third of the break-even rate of 0.340. It is not silent -- it is the opposite failure, sending 22 notifications a week at a click rate that cannot pay for them.
2. **It is beaten by a fixed 18:00 schedule on four of the five archetypes.** A learning algorithm losing to a hard-coded daily alarm is the single most informative result in this

notebook.

3. **It is the only method that drives meaningful churn.** An opt-out rate of 0.114 on Housewife means roughly one simulated user in nine quit permanently -- a cost the agent never learns to avoid.

Key findings and their implications

Finding 1 — the task is sequential, and a contextual bandit cannot represent that. LinUCB maximises *immediate* expected reward given the current context. But the dominant cost of a send, fatigue, is not paid immediately: it is paid over the following hours through the decay term λ. The click that a send produces is credited to that send; the fatigue it causes is charged to whatever action happens to follow. LinUCB therefore systematically under-estimates the true cost of sending, and over-sends by construction. No amount of tuning α repairs this, because the deficiency is in the problem formulation rather than in the parameters.

Finding 2 — this is the empirical justification for the rest of the project. LinUCB is the strongest possible argument for why the remaining three notebooks use reinforcement learning rather than bandits. The comparison is not RL-versus-nothing; it is RL versus a competent bandit that has the same features, the same budget and the same evaluation protocol, and still fails because it cannot propagate a delayed cost backwards in time.

Finding 3 — the belief state is necessary but not sufficient. Section 10 showed the belief-state encoding pays for itself relative to the raw encoding. That gain is real and it is still not enough: better features cannot substitute for a value function that looks ahead.

Implication for deployment. A bandit-based notification engine will appear to work in offline A/B evaluation, where fatigue has not yet accumulated, and will degrade over weeks in production as the fatigue it cannot model builds up. The churn figure above is what that degradation looks like.

How to read a reward of `0.00`

`0.00` is not a missing value and not a crash. It is exactly what an agent earns by never sending: no clicks are collected, so no click reward is paid, and no send or fatigue cost is incurred either. Because three of the five archetypes score *negatively* under the Fixed-18:00 baseline, silence genuinely outscores the industry-standard schedule on those users. Any agent that discovers this is not malfunctioning -- it has found a real, and uncomfortable, local optimum.

The economics that govern every result here

A send is worth making only when its click probability clears the break-even click-through rate

$$\mathrm{CTR}_{\text{break-even}} \;=\; \frac{W_{\text{send}} + W_{\text{fat}} \cdot \kappa/(1-\lambda)}{R_{\text{click}}} \;=\; 0.340$$

The $\kappa/(1-\lambda)$ term is the whole story: because fatigue decays geometrically rather than instantly, one send commits the agent to a stream of future fatigue penalties. At $\gamma = 0.99$ over a 168-step episode that discounted tail is worth roughly a hundred steps of debt while the click pays once, which is why silence can dominate. Every finding above is a consequence of that single inequality.